

Tong Xiao

Jingbo Zhu

Natural Language Processing

Neural Networks and Large Language Models

NATURAL LANGUAGE PROCESSING LAB

NORTHEASTERN UNIVERSITY

&

NIUTRANS RESEARCH

October 5, 2026

Copyright © 2021-2026 Tong Xiao and Jingbo Zhu

NATURAL LANGUAGE PROCESSING LAB, NORTHEASTERN UNIVERSITY
&
NIUTRANS RESEARCH

<https://github.com/NiuTrans/NLPBook>

<https://niutrans.github.io/NLPBook>

Licensed under the Creative Commons Attribution-NonCommercial 4.0 Unported License (the “License”). You may not use this file except in compliance with the License. You may obtain a copy of the License at <http://creativecommons.org/licenses/by-nc/4.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

October 5, 2026

Chapter 2

Foundations of Neural Networks

Artificial neural networks (or **neural networks**, or **neural nets** for short) are powerful machine learning tools that have advanced the state-of-the-art in NLP in recent years. Although the history of neural networks can be traced back to the 1940s [McCulloch and Pitts, 1943], for a long time, they did not consistently outperform other machine learning counterparts. This changed around 2006 when novel ideas were developed to train **deep neural networks** [Hinton et al., 2006; Hinton, 2007]. These methods have since become known as **deep learning**. To date, deep learning has undoubtedly become one of the most active and influential areas in artificial intelligence, benefiting not only from “deep” model architectures but also from numerous techniques that help to train and deploy such models.

In this chapter, we will present the fundamental concepts of neural networks and deep learning. While it does not focus on the latest cutting-edge research, this chapter covers several important concepts and techniques widely used in implementing neural systems. These include fundamental model architectures, training and regularization methods, unsupervised learning techniques, and auto-encoders. We will also present an example of applying neural networks to the task of language modeling.

2.1 Multi-layer Neural Networks

To get started, we give a brief introduction to **single-layer perceptrons** and extend them to a more general case where multiple layers are stacked to form a more complex network.

2.1.1 Single-layer Perceptrons

Single-layer perceptrons (or **perceptrons** for short) may be the simplest neural networks that have been developed for practical use [Rosenblatt, 1957; Minsky and Papert, 1969]. Often, they are thought of as biologically-inspired models that transform some input to some output. A perceptron comprises a number of **neurons** connecting input and output variables. Figure 2.1 shows a perceptron where there is only one neuron. In this example, there are two real-valued variables x_1 and x_2 for input and a binary variable y for output. The neuron reads the input variables and determines which output value is chosen. This procedure is analogous to that of

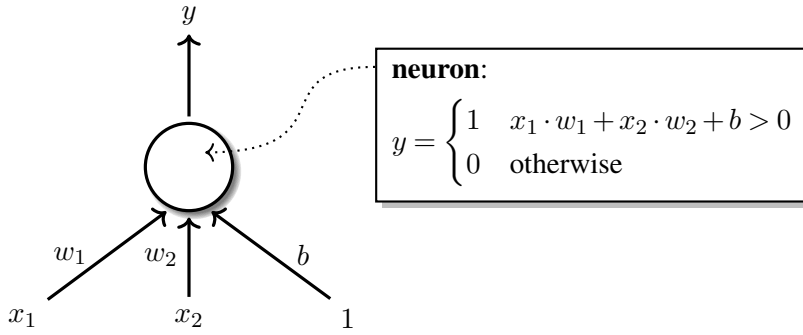


Figure 2.1: Illustration of a perceptron with two input variables $\{x_1, x_2\}$ and an output variable y . There are two weights $\{w_1, w_2\}$, each corresponding to an input variable. The output depends on the sum of the weighted input variables and the bias term b , e.g., $y = 1$ if $x_1 \cdot w_1 + x_2 \cdot w_2 + b > 0$, and $y = 0$ otherwise.

a biological neuron — it receives electrochemical inputs from other neurons and determines whether the electrochemical signal is passed along.

Mathematically, a perceptron can be described as a mapping function. Let $\mathbf{x}$ be a vector of input variables (i.e., a **feature vector**). An **affine transformation** of $\mathbf{x}$ is given by¹:

$$\begin{aligned} f(\mathbf{x}) &= \mathbf{x} \cdot \mathbf{w} + b \\ &= \sum_i x_i \cdot w_i + b, \end{aligned} \quad (2.1)$$

where $\mathbf{w}$ is a weight vector and b is a bias term. Then, a standard perceptron can be defined as:

$$\begin{aligned} y &= \psi(f(\mathbf{x})) \\ &= \begin{cases} 1 & f(\mathbf{x}) > 0 \\ 0 & \text{otherwise} \end{cases}, \end{aligned} \quad (2.2)$$

where $\psi(\cdot)$ is a binary **step function**. The function $\psi(\cdot)$ is also known as an activation function. This links the perceptron to the classification models discussed in Chapter 1. In other words, Eq. (2.2) acts as a classifier itself: $f(\mathbf{x})$ is a discriminant function defined on each input $\mathbf{x}$, followed by an activation function $\psi(\cdot)$ used to produce the desired output².

When there are two or more neurons, we can group these neurons into a **layer**. As shown in Figure 2.2, all the neurons in a layer receive signals from the same input feature vector but are weighted in different ways. The output of the layer is a new feature vector, each entry of which corresponds to a neuron. More formally, taking $\psi(\cdot)$ and $f(\cdot)$ as vector functions, the

¹In mathematics, a **linear transformation** maps each vector $\mathbf{v}$ in a space to $f(\mathbf{v})$ in another space, such that for any vectors $\mathbf{x}$ and $\mathbf{y}$, and scalars α and β , we have $f(\alpha\mathbf{x} + \beta\mathbf{y}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{y})$. An affine transformation is a linear transformation followed by a translation, often written in the form $f(\mathbf{x}) + \mathbf{b}$.

²Since the decision boundary is linear, the perceptron is considered a linear classifier, even though the step function itself is non-linear.

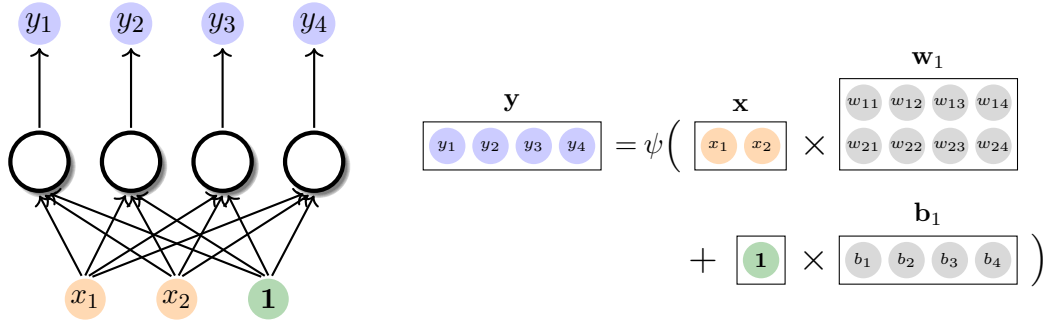


Figure 2.2: Illustration of a single-layer perceptron involving four neurons. All these neurons receive information from the input variables $\{x_1, x_2\}$. The perceptron describes a process where 1) we first transform the input vector of variables by an affine transformation $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + \mathbf{b}$; and 2) compute the output by feeding $f(\mathbf{x})$ into the activation function $\psi(\cdot)$.

mathematical form of the single-layer perceptron is given by the equations:

$$\mathbf{y} = \psi(f(\mathbf{x})), \quad (2.3)$$

$$f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} + \mathbf{b}, \quad (2.4)$$

where $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, $\mathbf{w} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^n$.

Another note on the activation function: the step function, though extensively used, is not the only form of the activation function. There are many different ways to perform activation. For example, we can use the Softmax function if we want a probability distribution-like output; we can use the Sigmoid function if we want a monotonic, continuous, easy-to-optimize output; we can use the ReLU function if we want a ramp-shaped output. Table 2.1 shows several commonly used activation functions. Note that, although a layer of neurons equipped with these activations can be loosely called a single-layer perceptron, it can be categorized as a more general concept, called a **single-layer neural network**. If not specified otherwise, we will use the term *single-layer neural network* throughout this book.

2.1.2 Stacking Multiple Layers

A next obvious step is to create a neural network comprising multiple layers. To do this, we simply stack multiple layers to form a **multi-layer neural network**. Figure 2.3 illustrates an example. In this architecture, the output of every neuron in a given layer is connected to all neurons in the subsequent layer. So the network is **fully connected**. Essentially, a multi-layer neural network describes a composition of functions. For example, we can formulate the neural network in Figure 2.3 as a complex function formed by the composition of several simpler functions:

$$\mathbf{y} = \text{Softmax}(\text{Sigmoid}(\text{ReLU}(\mathbf{x} \cdot \mathbf{w}_1) \cdot \mathbf{w}_2) \cdot \mathbf{w}_3 + \mathbf{b}_3), \quad (2.5)$$

where $\mathbf{w}_1 \in \mathbb{R}^{3 \times 4}$, $\mathbf{w}_2 \in \mathbb{R}^{4 \times 3}$, $\mathbf{w}_3 \in \mathbb{R}^{3 \times 3}$, and $\mathbf{b}_3 \in \mathbb{R}^3$ are parameters.

Name	Formula (for entry i of a vector)
Identity	$y_i = s_i$
Binary Step	$y_i = \begin{cases} 1 & s_i > 0 \\ 0 & s_i \leq 0 \end{cases}$
Hyperbolic Tangent	$y_i = \frac{\exp(s_i) - \exp(-s_i)}{\exp(s_i) + \exp(-s_i)}$
Hard Tangent	$y_i = \begin{cases} 1 & s_i > 1 \\ s_i & -1 \leq s_i \leq 1 \\ -1 & s_i < -1 \end{cases}$
Sigmoid (Logistic)	$y_i = \frac{1}{1 + \exp(-s_i)}$
ReLU (Rectified Linear Unit)	$y_i = \begin{cases} s_i & s_i > 0 \\ 0 & s_i \leq 0 \end{cases}$
Softplus	$y_i = \ln(1 + \exp(s_i))$
Gaussian	$y_i = \exp\left(-\frac{1}{2} \cdot \frac{(s_i - \mu_i)^2}{\sigma_i^2}\right)$
Softmax	$y_i = \frac{\exp(s_i)}{\sum_{i'=1}^n \exp(s_{i'})}$
Maxout	$y_i = \max(s_1, \dots, s_n)$

Table 2.1: Activation functions ($\mathbf{y} = \psi(\mathbf{s})$, where $\mathbf{s}, \mathbf{y} \in \mathbb{R}^n$). All these functions are vector functions. We show formulas for entry i of the input and output vectors. μ_i and σ_i^2 are the mean and variance respectively.

Typically, the **depth** of a neural network (sometimes referred to as **model depth**) is measured by the number of its layers. For example, treating the input vector as an initial layer, the depth of the example network in Figure 2.3 is 4. A related concept is **model width**, which is defined per layer rather than across the entire network. A common measure for the width of a layer is the number of neurons it contains. For example, the width of the output layer in Figure 2.3 is 3. If all layers of a neural network share the same width n , we can simply state that the overall model width is n . Both model depth and model width have important implications for the representational capacity of the resulting neural network. For example, it has been mathematically proven that even a neural network with just two layers of neurons and a Sigmoid activation function can approximate any continuous function [Cybenko, 1989]. However, for more complex tasks, increasing the depth of the network generally yields significant performance improvements.

Stacking layers results in a very common kind of neural network — **feed-forward neural networks (FFNNs)**. These networks are called “feed-forward” because there are no cycles in connections between layers and all the data moves in one direction. As we will see throughout this book, many modern neural networks are feed-forward in nature, though a few notable exceptions will be discussed in Section 2.3.

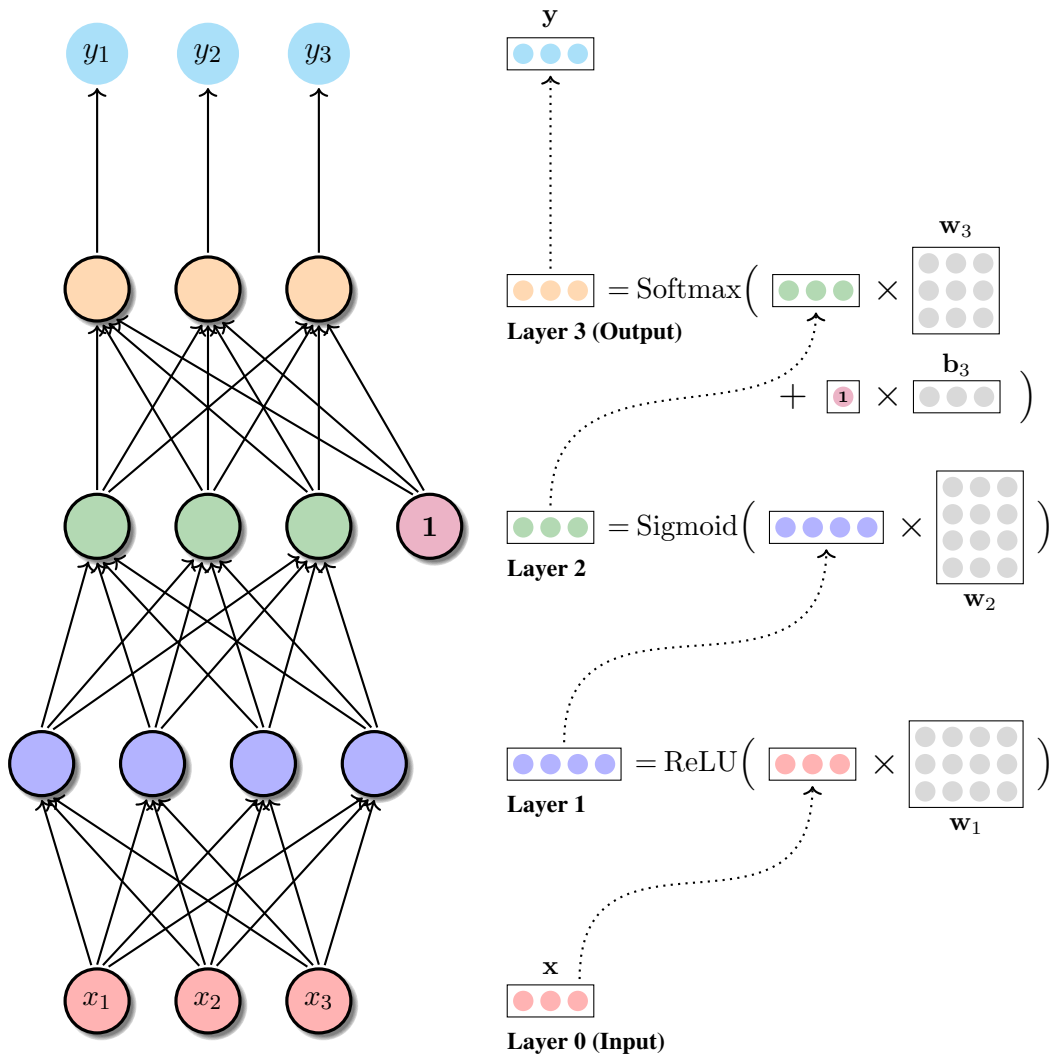


Figure 2.3: Illustration of a multi-layer neural network. The input layer consists of three variables $\{x_1, x_2, x_3\}$. These variables are fully connected to all neurons of layer 1. The output of layer 1 is a new vector $\mathbf{h}_1 = \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_1)$. It is then fully connected to layer 2, performing the mapping $\mathbf{h}_2 = \text{Sigmoid}(\mathbf{h}_1 \cdot \mathbf{w}_2)$. Its output $\mathbf{h}_2$ is fed into layer 3, which generates the final output $\mathbf{y} = \text{Softmax}(\mathbf{h}_2 \cdot \mathbf{w}_3 + \mathbf{b}_3)$. The parameters of this neural network are $\mathbf{w}_1$, $\mathbf{w}_2$, $\mathbf{w}_3$ and $\mathbf{b}_3$.

2.1.3 Computation Graphs

Computation graphs are a common way of representing neural networks. As graphs in mathematics, a computation graph is made up of nodes and edges between nodes. Each node represents either a mathematical operation or a variable, and each edge represents the data flow from one node to another. Therefore, computation graphs are directed³. Consider, for example,

³While a number of machine learning models can be represented as undirected computation graphs, they are not the focus of this book.

three functions:

$$\mathbf{y} = \mathbf{x} + \mathbf{w}, \quad (2.6)$$

$$\mathbf{y} = \text{Softmax}(\mathbf{x} \cdot \mathbf{w} + \mathbf{b}), \quad (2.7)$$

$$\mathbf{y} = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) - \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2). \quad (2.8)$$

Figure 2.4 shows the computation graphs of these functions. From a parsing perspective, all neural networks can be viewed as mathematical expressions. A computation graph is therefore the representation of the result when parsing a mathematical expression. In this way, each node of the graph yields a sub-expression, and the root node yields the whole expression.

In a computation graph, a node can be connected to multiple nodes beneath it and/or above it. This enables the reuse of sub-graphs in representing complex functions. For example, in Eq. (2.8), the variable $\mathbf{x}$ is used twice and the corresponding node has two outgoing edges. In fact, organizing neural networks into computation graphs resembles the compositional nature of neural networks — typically, a large network is built by composing small networks. Take Eq. (2.8) as an instance. It can be rewritten as a system of three equations:

$$\mathbf{y} = \mathbf{h}_1 - \mathbf{h}_2, \quad (2.9)$$

$$\mathbf{h}_1 = \text{Sigmoid}(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1), \quad (2.10)$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{x} \cdot \mathbf{w}_2). \quad (2.11)$$

In the composition operation, the nodes of $\mathbf{h}_1$ and $\mathbf{h}_2$ in Eq. (2.9) are replaced by the graphs of Eqs. (2.10-2.11).

The main use of computation graphs is in executing the function. This is exactly the same thing as predicting the output of a neural network. The method is quite simple. First, the nodes of the graph are topologically sorted such that they are placed in an order consistent with the information flow. Then, given the values that are fed into the input nodes, the graph is traversed in a way that we compute the output of each node and forward it to its parent nodes. The final result is obtained from the output node. This procedure is typically called a **forward pass**. A forward pass can be efficient, as every node only needs to be visited once and its output can be reused by multiple nodes without the need of recomputing the result. Moreover, a forward pass can be optimized by reconstructing the graph. This can develop the reuse idea a bit more and avoid unnecessary computation and memory consumption.

Another use of computation graphs is to compute gradients automatically. In training neural networks, it is in general required to compute the partial derivatives of the loss function $\mathcal{L}$ with respect to every weight matrix ($\mathbf{w}$) and every bias term ($\mathbf{b}$), such as $\frac{\partial \mathcal{L}}{\partial \mathbf{w}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{b}}$. Before seeing how these partial derivatives are used in updating a model (see Section 2.4.1), though, we first give an idea of computing derivatives in a computation graph. For example, consider the function below:

$$\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{w}_1 + \mathbf{b}_1) \cdot \mathbf{w}_2. \quad (2.12)$$

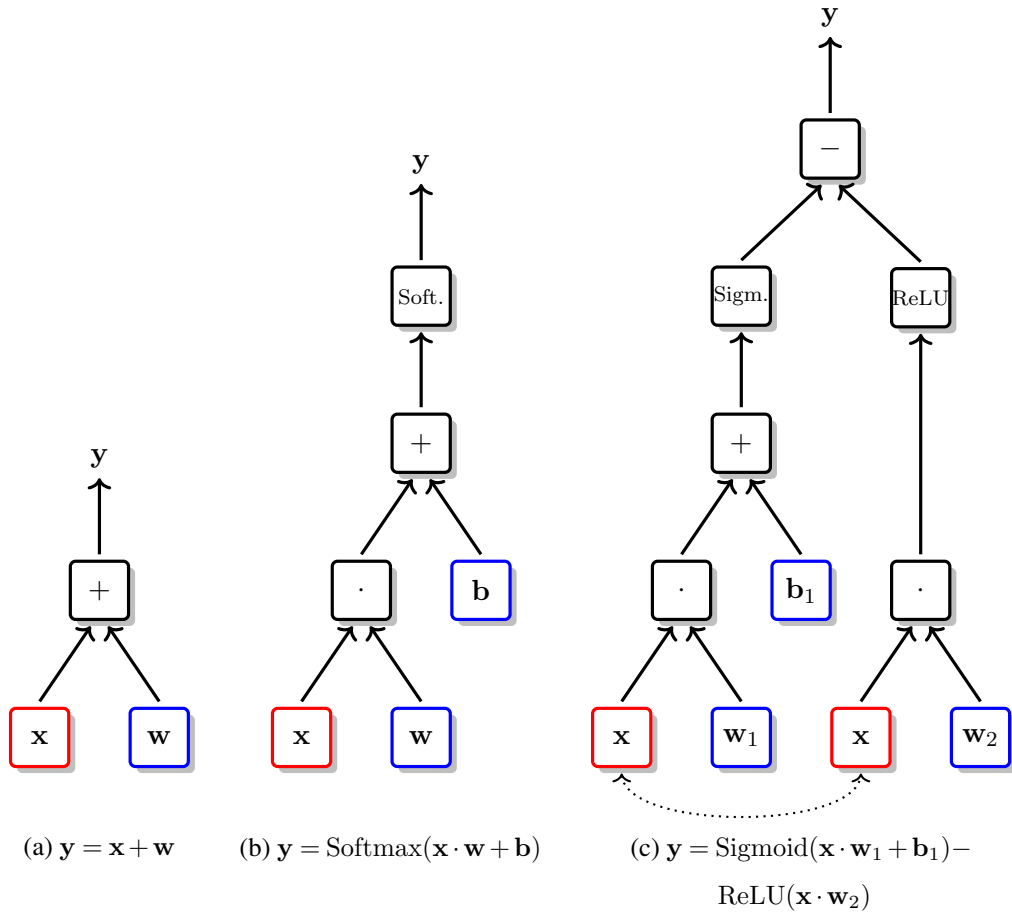


Figure 2.4: Computation graphs of three example neural networks. The black boxes represent the mathematical operations, and the colored boxes represent the variables. A mathematical operation node has incoming edges from other nodes, and each of these nodes can be treated as an argument of the operation. For example, in sub-figure (a), the addition node has two child nodes labeled with x and w respectively. This node reads the output of the nodes x and w , and generates the output $y = x + w$. Larger graphs exhibit more complex data flows. In sub-graph (b), the output of the dot node (i.e., $x \cdot w$) is passed along the edge to the addition node. Then, the addition node computes the sum of $x \cdot w$ and b as its output. We can repeat the same process over all the mathematical operation nodes in a bottom-up manner, and get the final result of computing the whole expression out of the top-most node.

To obtain $\frac{\partial \mathcal{L}}{\partial w_1}$, $\frac{\partial \mathcal{L}}{\partial b_1}$ and $\frac{\partial \mathcal{L}}{\partial w_2}$, it is natural to use the chain rule of differentiation. For example, for a composite function $y = p(q(x))$, the formula of the chain rule is given as:

$$\frac{\partial y}{\partial x} = \frac{\partial p}{\partial q} \cdot \frac{\partial q}{\partial x}. \quad (2.13)$$

However, the analytic formula of a derivative based on Eq. (2.13) would result in a lengthy equation. Instead, we can decompose a complex function into several functions, each standing

for an operation. Then, Eq. (2.12) can be rewritten as:

$$\mathbf{y} = \mathbf{h}_1 \cdot \mathbf{w}_2, \quad (2.14)$$

$$\mathbf{h}_1 = \psi(\mathbf{h}_2), \quad (2.15)$$

$$\mathbf{h}_2 = \mathbf{h}_3 + \mathbf{b}_1, \quad (2.16)$$

$$\mathbf{h}_3 = \mathbf{x} \cdot \mathbf{w}_1. \quad (2.17)$$

All these variables can be better understood intuitively from a computation graph: each variable is a node of the graph, and nodes are connected by algebraic operations and function compositions. Taking Eq. (2.13) and some basic knowledge of calculus, we compute the derivatives of the variables, as follows:

$$\text{node 1:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{y}} = \delta_{\mathbf{y}}, \quad (2.18)$$

$$\text{node 2:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \mathbf{w}_2^\top, \quad (2.19)$$

$$\text{node 3:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{w}_2} = \mathbf{h}_1^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{y}}, \quad (2.20)$$

$$\text{node 4:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \odot \psi'(\mathbf{h}_2), \quad (2.21)$$

$$\text{node 5:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2}, \quad (2.22)$$

$$\text{node 6:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_2}, \quad (2.23)$$

$$\text{node 7:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3} \cdot \mathbf{w}_1^\top, \quad (2.24)$$

$$\text{node 8:} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{w}_1} = \mathbf{x}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}_3}, \quad (2.25)$$

where $\delta_{\mathbf{y}}$ is the derivative of the loss with respect to the model output. $\delta_{\mathbf{y}}$ depends on the choice of the loss function, e.g., if we use the squared loss $\mathcal{L} = \frac{1}{2}(\mathbf{y} - \mathbf{y}_{\text{gold}})^2$, where $\mathbf{y}_{\text{gold}}$ is the ground-truth, then $\delta_{\mathbf{y}} = \mathbf{y} - \mathbf{y}_{\text{gold}}$. The above process is essentially a **backward pass**, as the gradients are passed in a top-down fashion. Another name for this is **backpropagation**. It has been the de facto standard for training deep neural networks. For a better understanding of how forward and backward passes work, Figure 2.5 shows two running examples.

2.2 Example: Neural Language Modeling

Language modeling is a fundamental NLP task that estimates a probability distribution over sequences of words. Given a sequence of m words $w_1 \cdots w_m$, the joint probability

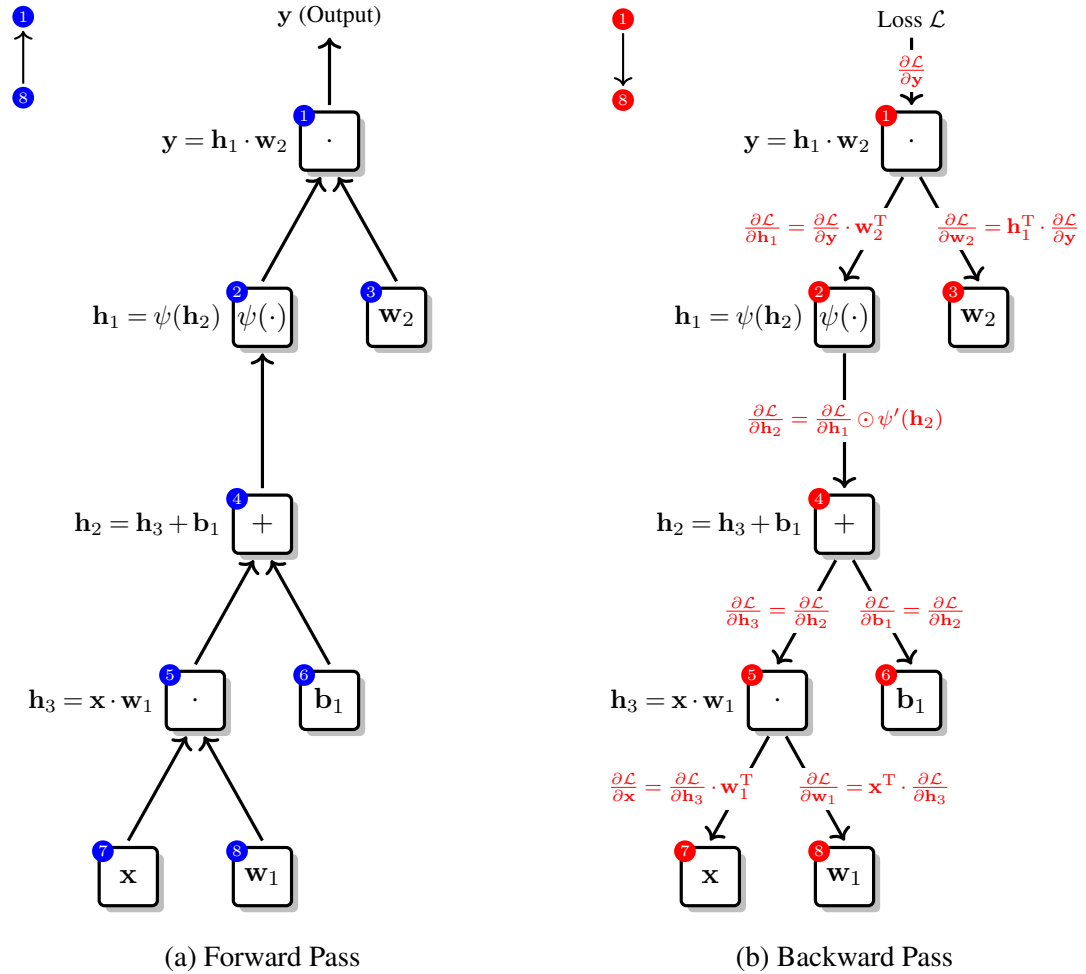


Figure 2.5: Illustration of the forward pass and backward pass for an example computation graph. In the forward pass (left), the nodes are visited in an order from the input to the output, e.g., from node 8 to 1. On each node, we execute the corresponding function, such as addition, to generate the output, which is then consumed by the subsequent nodes. In contrast, in the backward pass (right), the nodes are visited in the reverse order, e.g., from node 1 to 8. During this process, we pass the gradient of the loss (or error) from the output to the input, that is, for each node, we compute the gradient at the input point of the node by using the chain rule, given the gradient at the output point of the node.

$\Pr(w_1, \dots, w_m)$ is given by the equation:

$$\Pr(w_1, \dots, w_m) = \prod_{i=1}^m \Pr(w_i | w_1, \dots, w_{i-1}). \quad (2.26)$$

As such, the language modeling problem is framed as predicting the next word given all previous context words. A straightforward method to model $\Pr(w_i | w_1, \dots, w_{i-1})$ is to condition

the prediction on a context window covering a fixed number of preceding words, as follows:

$$\Pr(w_i|w_1, \dots, w_{i-1}) \approx \Pr(w_i|w_{i-n+1}, \dots, w_{i-1}), \quad (2.27)$$

where n is the window size. One way to estimate this probability is the **n -gram language modeling** approach: we compute the relative frequency for each n -gram $w_{i-n+1} \dots w_i$, i.e., $\Pr(w_i|w_{i-n+1}, \dots, w_{i-1}) = \frac{\text{count}(w_{i-n+1} \dots w_i)}{\text{count}(w_{i-n+1} \dots w_{i-1})}$. Although n -gram language models dominated the NLP field for decades, they require massive tables to store all n -gram probabilities. In consequence, as the training corpus grows and longer contexts are considered, the data becomes exceedingly sparse. This is a classic instance of the curse of dimensionality.

Here we consider neural networks in addressing the language modeling problem [Bengio et al., 2000; 2003]. Unlike n -gram models, which operate in a discrete space requiring an exponentially large number of distinct feature vectors, **neural language models** generalize in a continuous space by encoding words as dense, low-dimensional real-valued vectors. In particular, a feed-forward network is utilized here to predict the likelihood of w_i occurring given the context $w_{i-n+1} \dots w_{i-1}$.

Figure 2.6 presents the architecture of the **feed-forward neural network based language model (FFNNLM)**. The input is the context words $w_{i-n+1} \dots w_{i-1}$. Each is a discrete variable taking values from a vocabulary V . Since the neural network operates on vectors, all words are vectorized as **one-hot representations**. In this case, the word $w = V_k$ is a $|V|$ -dimensional vector in which entry k is 1 and other entries are all 0. For example, consider a vocabulary $V = \{\text{"I"}, \text{"you"}, \text{"he"}, \text{"she"}, \text{"they"}\}$. The one-hot representation of “you” is expressed as

$$\mathbf{1}_{\text{you}} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \end{bmatrix}. \quad (2.28)$$

Although one-hot vectors uniquely identify words, they fail to capture semantic relationships (i.e., the expectation that similar words should be situated close to one another in the vector space). By relaxing these indicator-based representations into continuous, real-valued vectors, we can capture word relationships by computing similarities between these vectors. To this end, an effective technique is to transform one-hot representations into **distributed representations**. More formally, let $\mathbf{x}$ be a one-hot vector of a word w . Its distributed representation is a real-valued vector, given by:

$$\mathbf{e} = \mathbf{x} \cdot \mathbf{C}, \quad (2.29)$$

where the representation $\mathbf{e}$ is a vector $\in \mathbb{R}^{d_e}$, and d_e is the dimensionality of the representation. Each dimension of $\mathbf{e}$ can be viewed as some countable aspect of the word, though it is not required to be interpreted linguistically. $\mathbf{C}$ is a $|V| \times d_e$ matrix, of which the k -th row corresponds to the vector for V_k . Hence, $\mathbf{x} \cdot \mathbf{C}$ is to “select” a row from $\mathbf{C}$. For example, given

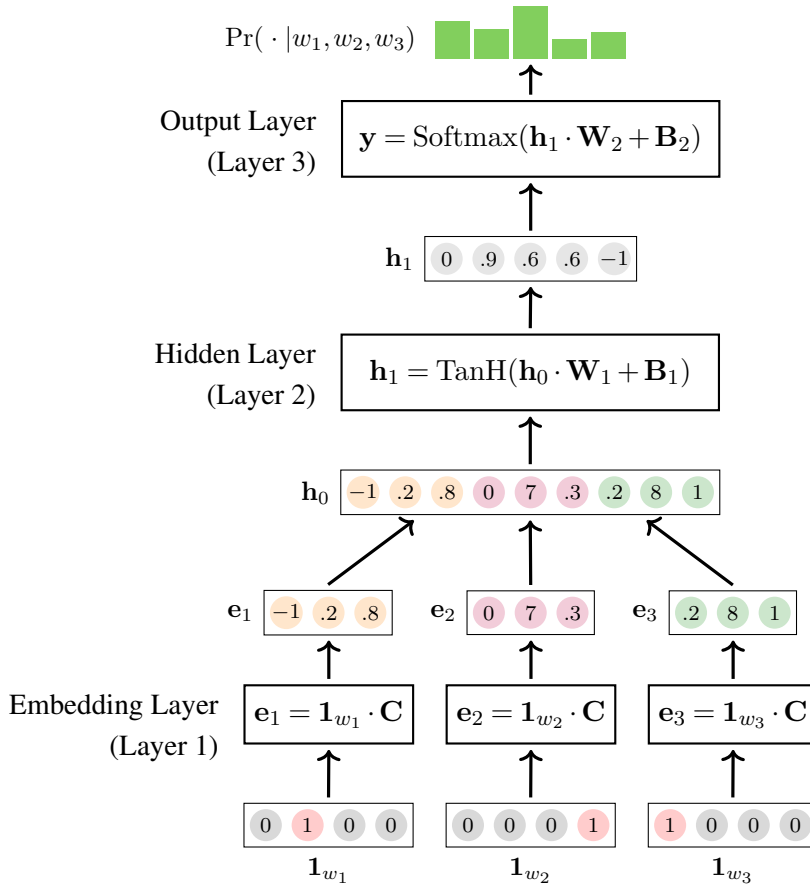


Figure 2.6: Illustration of a neural language model [Bengio et al., 2003]. The boxes represent the layers of the neural network. The input is three context words in their one-hot representations $\{\mathbf{1}_{w_1}, \mathbf{1}_{w_2}, \mathbf{1}_{w_3}\}$, and the output is the probability distribution of the next word $\text{Pr}(w_4 | w_1, w_2, w_3)$. First, an embedding layer is used to map each word into the distributed representation (i.e., the word embedding). The embeddings of these words are concatenated to form a bigger vector $\mathbf{h}_0$ such that the concatenated vector encodes all input information. Then, $\mathbf{h}_0$ is taken as the input to a normal layer that performs the mapping $\mathbf{h}_1 = \text{TanH}(\mathbf{h}_0 \cdot \mathbf{W}_1 + \mathbf{B}_1)$. The final layer reads $\mathbf{h}_1$ and produces a distribution over the vocabulary, i.e., $\mathbf{y} = \text{Softmax}(\mathbf{h}_1 \cdot \mathbf{W}_2 + \mathbf{B}_2)$ where $y_k = \text{Pr}(V_k | w_1, w_2, w_3)$.

$\mathbf{C} \in \mathbb{R}^{5 \times 3}$, the distributed representation of “you” is given by:

$$\begin{aligned}
 e(\text{“you”}) &= \mathbf{1}_{\text{you}} \cdot \mathbf{C} \\
 &= \begin{bmatrix} 0 & \textcolor{red}{1} & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 73 & 12 & 0.1 \\ \textcolor{red}{12} & \textcolor{red}{0.5} & \textcolor{red}{18} \\ 37 & 0.7 & 28 \\ 61 & 0.4 & 23 \\ 62 & 11 & 0.4 \end{bmatrix} \\
 &= \begin{bmatrix} 12 & 0.5 & 18 \end{bmatrix}.
 \end{aligned} \tag{2.30}$$

Eq. (2.29) introduces the foundational concept of learning word representations, which has driven massive advancements in NLP. Typically, the vector $\mathbf{e}$ is called a **word embedding**, and the parameter matrix $\mathbf{C}$ is called the **embedding matrix**. While many methods exist for learning word embeddings, we will not focus on them in this chapter. The reader is referred to Chapter 3 for a detailed discussion on this topic.

To encode the context words $\{w_{i-n+1}, \dots, w_{i-1}\}$, a simple method is to concatenate the word embeddings $\{\mathbf{e}_{i-n+1}, \dots, \mathbf{e}_{i-1}\}$ as a new vector $\mathbf{h}_0$:

$$\mathbf{h}_0 = [\mathbf{e}_{i-n+1}, \dots, \mathbf{e}_{i-1}].$$

The next part of the model is a 2-layer feed-forward neural network. The first layer, called a **hidden layer**, is a standard layer of neurons, followed by the hyperbolic tangent activation function. The layer produces a d_h -dimensional vector:

$$\mathbf{h}_1 = \text{TanH}(\mathbf{h}_0 \cdot \mathbf{W}_1 + \mathbf{B}_1). \quad (2.31)$$

The second layer is the output layer. It produces a distribution over V . This can be formulated as:

$$\Pr(\cdot | w_{i-n+1}, \dots, w_{i-1}) = \text{Softmax}(\mathbf{h}_1 \cdot \mathbf{W}_2 + \mathbf{B}_2). \quad (2.32)$$

The parameters of the model are $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$, $\mathbf{W}_1 \in \mathbb{R}^{(n-1)d_e \times d_h}$, $\mathbf{B}_1 \in \mathbb{R}^{d_h}$, $\mathbf{W}_2 \in \mathbb{R}^{d_h \times |V|}$, and $\mathbf{B}_2 \in \mathbb{R}^{|V|}$. A popular way to optimize these parameters is to minimize the cross-entropy loss via gradient descent. Additionally, training can be improved via regularization. These methods will be discussed in Sections 2.4 and 2.5.

A few remarks on the neural language model: first, by using distributed feature vectors, “senses” can be shared in part by different words. This enables learnable word senses by which the similarity between words is implicitly considered. An advantage of such a model is that a small change in word vectors would not lead to a big change in the result. For example, suppose we have seen “grapes are fruits” many times but have never seen “peaches are fruits”. If the embeddings for “grapes” and “peaches” are close, the model can logically infer that the phrase “peaches are fruits” is highly probable. This differentiates neural language models greatly from n -gram language models where different surface forms are treated as completely independent entities.

Second, the dense representation of words results in a smaller model. For example, a common setting of d_e and d_h is less than 1000, keeping the total number of parameters manageable. By contrast, the size of an n -gram language model increases by a factor of $|V|$ as n increases. For example, there will be a huge table of probabilities for a common vocabulary if n is larger than 3.

Third, the neural language model is computationally expensive because of the heavy use of vector and matrix operations, such as matrix multiplication. This is a common problem with most deep neural network-based systems. A common solution is to break the computation problem into independent sub-problems so that these sub-problems can be handled in parallel.

At a lower level, one can use GPUs or other parallel computing devices to speed up linear algebra operators. At a higher level, one can distribute parts of the model or parts of the data to multiple devices for model-level or task-level speed-ups.

2.3 Basic Model Architectures

We now describe, in more detail, several basic building blocks for neural networks. They are widely used in developing state-of-the-art neural models in NLP.

2.3.1 Recurrent Units

Recurrent neural networks (RNNs) are a class of neural networks that read and/or produce sequential data or time-series data. As with a feed-forward neural network, an RNN comprises layers of neurons and connections between neurons [Hopfield, 1984; Rumelhart et al., 1986; Williams and Zipser, 1989; Elman, 1990]. Some of the neurons are used as a “memory” that keeps the state of the problem when the processing moves on along a sequence of signals. As a result, it is straightforward to use RNNs to deal with variable-length problems, such as machine translation and speech recognition.

The main idea behind RNNs is to repeatedly utilize a **recurrent unit** (or **recurrent cell**) to compute the output at each position of an input sequence. To be more precise, given a sequence of vectors $\mathbf{x}_1 \cdots \mathbf{x}_m$, a standard **recurrent unit** can be described as a function $\text{RNN}(\cdot)$ that consumes an input $\mathbf{x}_i$ and a state $\mathbf{s}_{i-1}$ at each time step and generates a new state $\mathbf{s}_i$, given by:

$$\mathbf{s}_i = \text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i). \quad (2.33)$$

The state $\mathbf{s}_i$ can be viewed as a “memory” that summarizes past information and is updated as new data arrives. See Figure 2.7 (a) for visualization of Eq. (2.33). The circle here indicates the reuse of the recurrent unit. This can be understood by rewriting Eq. (2.33) in a sequence of calls of the function $\text{RNN}(\cdot)$:

$$\begin{aligned} \text{RNN}(\mathbf{s}_{i-1}, \mathbf{x}_i) &= \text{RNN}(\text{RNN}(\mathbf{s}_{i-2}, \mathbf{x}_{i-1}), \mathbf{x}_i) \\ &= \text{RNN}(\text{RNN}(\text{RNN}(\mathbf{s}_{i-3}, \mathbf{x}_{i-2}), \mathbf{x}_{i-1}), \mathbf{x}_i) \\ &= \dots \\ &= \text{RNN}(\text{RNN}(\dots (\text{RNN}(\mathbf{s}_0, \mathbf{x}_1), \mathbf{x}_2) \cdots \mathbf{x}_{i-1}), \mathbf{x}_i). \end{aligned} \quad (2.34)$$

Figure 2.7 (b) illustrates the structure of this expanded network. This formulation is typically referred to as the **unrolled** (or **unfolded**) structure of an RNN. Fundamentally, Figures 2.7 (a) and (b) represent the exact same mathematical operations. While a rolled RNN offers a compact and elegant mathematical form, an unrolled RNN provides a clearer visualization of the data flow through time. Therefore, we will use the unrolled representation of RNNs throughout this book.

Moreover, it is worth noting that an unrolled RNN can be viewed as a deep feed-forward

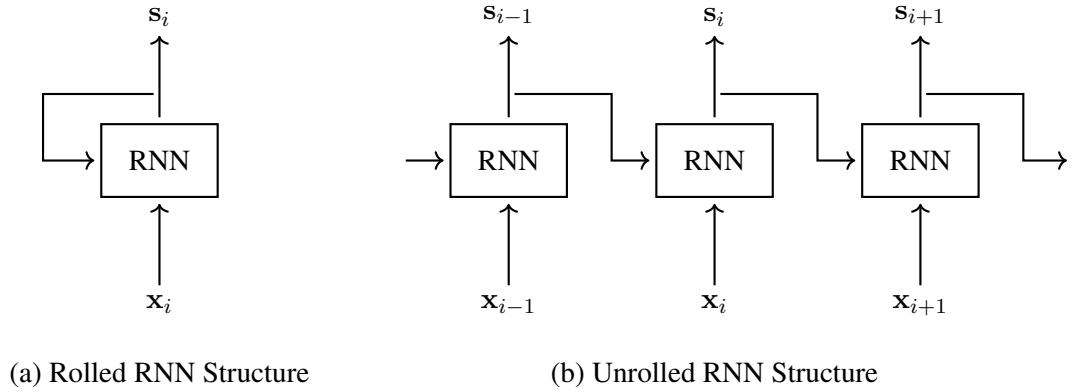


Figure 2.7: Example of RNN (rolled vs unrolled). An RNN unit reads the input at each time step i and the output at the last time step $i - 1$, and produces a new output s_i . As such we can reuse the same RNN unit to make predictions over a sequence of inputs (see sub-figure (a)): for each i , the current input $\mathbf{x}_i$ and last output s_{i-1} are consumed and mapped to the output that is fed into the same RNN unit for the processing at the next time step. A better way to visualize the RNN is to unroll it into a network with no cycles (see sub-figure (b)). The unrolled RNN can be regarded as a deep feed-forward neural network in that all RNN units share the same set of parameters.

neural network, where each application of the recurrent unit acts as a “layer” receiving information from the preceding “layer” (i.e., the previous time step). In this sense, an RNN is effectively a stack of layers extending horizontally across time. A benefit of this perspective is that one can utilize similar optimization techniques to train both architectures. For example, both RNNs and feed-forward networks rely on gradient-based optimization, though RNNs specifically employ a variant of backpropagation known as **backpropagation through time (BPTT)** to propagate errors backward through the unrolled computation graph.

There are many RNN variants, differing in how the function $\text{RNN}(\cdot)$ is defined. The simplest of these is to formulate $\text{RNN}(\cdot)$ as a single-layer neural network. Assume that s_{i-1} and $\mathbf{x}_i$ are in $\mathbb{R}^{d_h}$. The form of $\text{RNN}(\cdot)$ is given by:

$$\text{RNN}(s_{i-1}, \mathbf{x}_i) = \psi(s_{i-1} \cdot \mathbf{U} + \mathbf{x}_i \cdot \mathbf{V}), \quad (2.35)$$

where $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$ and $\mathbf{V} \in \mathbb{R}^{d_h \times d_h}$ are parameters. The common choices for the activation function ψ are $\text{Tanh}(\cdot)$, $\text{Sigmoid}(\cdot)$, $\text{ReLU}(\cdot)$, among others. Eq. (2.35) is a single-layer neural network because it has the same form as Eqs. (2.3-2.4):

$$\psi(s_{i-1} \cdot \mathbf{U} + \mathbf{x}_i \cdot \mathbf{V}) = \psi([s_{i-1}, \mathbf{x}_i] \cdot \mathbf{W}), \quad (2.36)$$

where $[s_{i-1}, \mathbf{x}_i]$ is the concatenation of s_{i-1} and $\mathbf{x}_i$, and $\mathbf{W} \in \mathbb{R}^{2d_h \times d_h}$ is the parameter matrix that is formed by $\begin{bmatrix} \mathbf{U} \\ \mathbf{V} \end{bmatrix}$.

In practice, RNNs are frequently used as sub-components within larger architectures. For example, the input to a recurrent unit could be a direct representation of raw data (such as word embeddings) or the output from another neural network module. Similarly, additional neural network layers are routinely stacked on top of recurrent units. In many real-world NLP systems, an output layer is stacked directly upon the hidden state s_i to project it into the desired output space for classification or prediction tasks.

2.3.2 Convolutional Units

Convolutional neural networks (CNNs) are another well-known class of neural networks [Waibel et al., 1989; LeCun et al., 1989]. In a biological sense, they are inspired by the human visual system: neurons react to stimuli in a restricted region of the visual field (known as the **receptive field**) [Hubel and Wiesel, 1959]. In CNNs, the receptive field describes the region in the input space that is involved in generating the output for a neuron. CNNs are therefore “partially connected” models in which each neuron only considers input features within a restricted region. This differentiates CNNs from fully connected feed-forward neural networks. In general, CNNs naturally capture the hierarchical nature of features describing data and scale more efficiently in terms of complexity.

While CNNs have many applications in processing 2D data, such as image classification, we discuss them here in a sequential data processing scenario to maintain a consistent treatment of the problem throughout this chapter. Typically, a CNN consists of a **convolutional layer**, a **pooling layer**, and, optionally, other types of layers. It begins with the convolutional layer, where the receptive field is defined by a set of **convolution kernels** or **filters**. A filter is a linear mapping function that convolves the input features in the receptive field to form a single output feature. For example, consider a sequence of numbers $x_1 \cdots x_m$. The filter ranging from position i to position $i + r - 1$ is defined to be:

$$\begin{aligned} v &= \text{Conv}(\mathbf{x}_{[i, i+r-1]}, \mathbf{W}) \\ &= \mathbf{x}_{[i, i+r-1]} \cdot \mathbf{W} \\ &= \sum_{k=0}^{r-1} x_{i+k} \cdot W_k, \end{aligned} \tag{2.37}$$

where r is the size of the receptive field, $\mathbf{x}_{[i, i+r-1]}$ is the sub-sequence $x_i \cdots x_{i+r-1}$, and $\mathbf{W} \in \mathbb{R}^r$ are the parameters of the filter. Then, a sequence of output features can be generated by moving the filter along the input sequence. Let stride be the distance between consecutive moves. The output for move i is then defined to be:

$$v_i = \text{Conv}(\mathbf{x}_{[\text{stride} \times i, \text{stride} \times i + r - 1]}, \mathbf{W}). \tag{2.38}$$

In this way, the convolutional layer transforms the input sequence $x_1 \cdots x_m$ to the output sequence $v_1 \cdots v_{\lfloor \frac{m}{\text{stride}} \rfloor}$ ⁴. A remark here is that the parameters $\mathbf{W}$ are shared across positions of the input sequence. This method is known as **parameter sharing** or **weight sharing**.

⁴ $\lfloor \cdot \rfloor$ stands for the floor function.

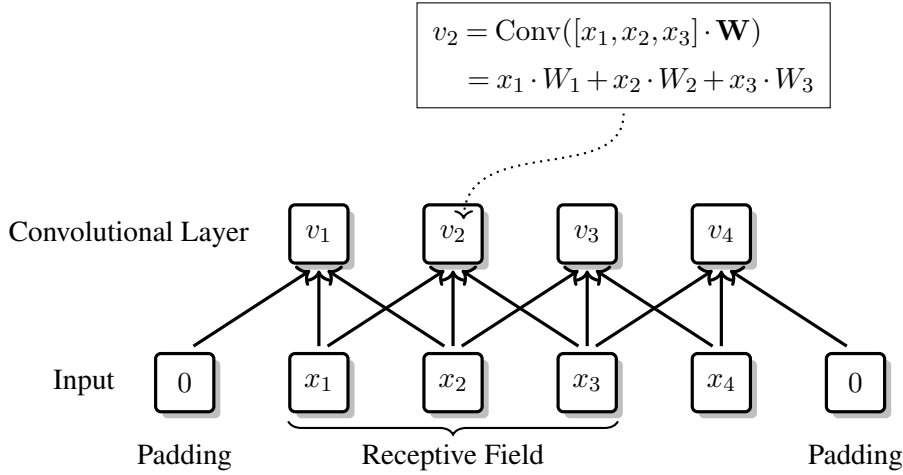


Figure 2.8: Illustration of convolution over a sequence of numbers $\{x_1, x_2, x_3, x_4\}$ ($r = 3$ and stride = 1). The receptive field defines the region in the input that is taken in computing the output. Here the receptive field has a size of 3, that is, the convolutional operation covers three consecutive numbers in the input sequence. The filter (or convolutional kernel) outputs a weighted sum of these numbers. Each time we slide the receptive field over the input, the filter generates a new output. As such, the output of the convolutional layer is a vector of numbers. Also, a padding number (i.e. 0) is added to each end of the sequence so that the convolution is feasible when the receptive field is incomplete.

Parameter sharing makes a CNN efficient because it requires fewer parameters than a feed-forward neural network given the same number of neurons.

A problem with the above formulation is that the use of the filter may not be tiled to fit the input sequence. For example, when $\text{stride} \times i + r - 1 > m$, the input of the filter is incomplete. A commonly used solution is **padding**. It simply sets the features outside the input sequence to a constant. For example, we can append dummy feature vectors to each end of the sequence so that all convolution operations are feasible. See Figure 2.8 for an example filter computed over a sequence of numbers. Note that the receptive fields of different filter applications may overlap. This is beneficial sometimes because it reduces information loss in feature representation when a low-level feature is used in forming multiple high-level features.

In addition, a convolutional layer can involve an activation function $\psi(\cdot)$ to perform some non-linear mapping on the filter output. Let $m_k = \lfloor \frac{m}{\text{stride}} \rfloor$ be the number of filter applications. The output of a convolutional layer is given by

$$\begin{bmatrix} h_1 & \cdots & h_{m_k} \end{bmatrix} = \psi(\begin{bmatrix} v_1 & \cdots & v_{m_k} \end{bmatrix}). \quad (2.39)$$

In general, a convolutional layer may not be restricted to a scalar-based input or a single filter. Often, we can take a vector as the representation of a token in the input sequence, and take a set of filters as feature extractors. To this end, we can adopt the same formulation as in Eqs. (2.37-2.39), but replace x_i , v_i and h_i with the vectorized counterparts.

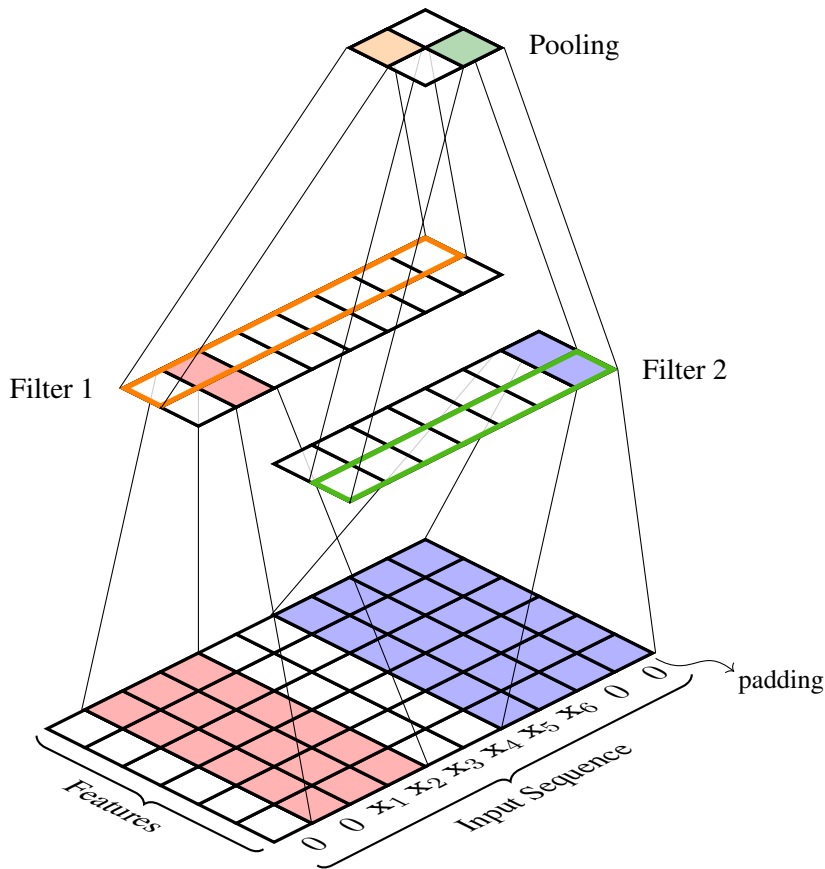


Figure 2.9: Example of CNN. There are two filters for the convolutional layer. The input is a sequence of 6 tokens represented in their feature vectors $\{x_1, \dots, x_6\}$. To tile the filters to fit the input sequence, two padding vectors are attached to each end of the sequence. When applying a filter, we map the feature vectors in the receptive field to a new feature vector. For example, for filter 1, the receptive field is a 6×3 rectangle in the input, and the output is a 2-dimensional feature vector. By sweeping the filter on the sequence, we obtain a sequence of feature vectors, e.g., a sequence of 8 feature vectors, each having 2 features. The pooling layer fuses features along the sequence. For example, performing the pooling on the output of filter 1 results in 2 fused features. The final output of the CNN is two 2-dimensional feature vectors.

A convolutional layer is typically followed by a pooling layer. Like convolution, pooling is a function that sweeps a filter on a sequence. But the pooling operation does not have any parameters. It can be instead thought of as an aggregation function that performs down-sampling on the input sequence. There are several ways to design a pooling function. One of the most common methods is **max pooling**, which outputs the maximum value in the receptive field. Another method is **average pooling** which outputs the averaged value over the receptive field. For a complete picture of how a CNN works, Figure 2.9 depicts a running example where convolutional and pooling operations are performed on a sequence of feature vectors via two filters.

2.3.3 Gate Units

In neural networks, a **gate** is used to decide how much information is passed along [Hochreiter and Schmidhuber, 1997]. Consider a standard RNN as an example. At each time step i , instead of directly passing the previous state $\mathbf{s}_{i-1} \in \mathbb{R}^{d_h}$ to the recurrent unit, it is often more effective to determine what portion of the information in $\mathbf{s}_{i-1}$ is relevant for the subsequent decision. In this context, we want $\mathbf{s}_{i-1}$ to function more like a true memory: as time progresses, certain information should be retained while other information is discarded.

A common approach to achieving this is to introduce a gating vector to control the scale of the data flow. Here, we reuse the notation from RNNs (see Section 2.3.1), but our description is general and applies to any architecture requiring such a mechanism. Let $\mathbf{z} \in [0, 1]^{d_h}$ be a gating vector, where $z_k = 0$ implies complete forgetting for dimension k , and $z_k = 1$ implies complete retention for dimension k . Applying $\mathbf{z}$ as a gate to $\mathbf{s}_{i-1}$ can be formulated as:

$$\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1}) = \mathbf{z} \odot \mathbf{s}_{i-1}, \quad (2.40)$$

where $\odot$ denotes the element-wise multiplication of two vectors. The operation $\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1})$ represents a filtered update to $\mathbf{s}_{i-1}$. The vector $\mathbf{z}$ can be referred to as an update gate, a forget gate, or a similarly functioning component. Alternatively, the gating function can be defined as:

$$\text{Gate}(\mathbf{z}, \mathbf{s}_{i-1}) = (1 - \mathbf{z}) \odot \mathbf{s}_{i-1}. \quad (2.41)$$

Eqs. (2.40) and (2.41) describe the same underlying mechanism but entail different interpretations for $\mathbf{z}$.

The central question is how to compute $\mathbf{z}$. A standard method is to define $\mathbf{z}$ as the output of an internal neural network layer. For example, in a recurrent unit, $\mathbf{z}$ can be defined as:

$$\mathbf{z} = \text{Sigmoid}(\mathbf{s}_{i-1} \cdot \mathbf{W}_1 + \mathbf{x}_i \cdot \mathbf{W}_2 + \mathbf{b}). \quad (2.42)$$

The use of the Sigmoid activation function ensures that the output values fall strictly within the range $[0, 1]$. Note that Eq. (2.42) describes a learnable gate. This incorporates the gating mechanism directly into the model, allowing it to be optimized via standard training procedures. Various gating architectures exist, several of which will be explored in detail in Chapter 4.

2.3.4 Normalization (Standardization) Units

A neural network works by transforming feature vectors layer by layer. While the multi-layer, multi-dimensional nature of neural networks enables the models to compute complex functions, it might lead to very different distributions of output activations across layers or features. This is a problem with deep neural networks because such models have to adapt to different distributions over different layers or different features [Ioffe and Szegedy, 2015]. Sometimes, as model parameters are initialized randomly in all layers and in all feature dimensions, some features are likely to take on disproportionately large values. In this case, the model would become biased toward those features with larger magnitudes.

A way to mitigate this issue is **normalization**, which standardizes an n -dimensional feature vector $\mathbf{s}$ as

$$\text{Normalize}(\mathbf{s}) = \boldsymbol{\alpha} \odot \frac{\mathbf{s} - \boldsymbol{\mu}}{\boldsymbol{\sigma} + \epsilon} + \boldsymbol{\beta}, \quad (2.43)$$

where $\boldsymbol{\mu} \in \mathbb{R}^n$ and $\boldsymbol{\sigma} \in \mathbb{R}^n$ are the mean and standard deviation of $\mathbf{s}$, respectively. ϵ is a small number used for numerical stability [Chiang and Cholak, 2022]. $\boldsymbol{\alpha} \in \mathbb{R}^n$ and $\boldsymbol{\beta} \in \mathbb{R}^n$ are the parameters of the normalization unit. A simple choice is $\boldsymbol{\alpha} = \mathbf{1}$ and $\boldsymbol{\beta} = \mathbf{0}$, whereas a more sophisticated method is to learn $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ together with other parameters.

We may implement Eq. (2.43) in several ways that differ in how to define $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$. Let us consider this for one dimension in $\mathbf{s}$, say s , in a general setting. Suppose that s is drawn from a set of feature values Ω_k . The mean and the standard deviation on Ω_k are then defined to be:

$$\mu_k = \frac{1}{|\Omega_k|} \sum_{s \in \Omega_k} s, \quad (2.44)$$

$$\sigma_k = \sqrt{\frac{1}{|\Omega_k|} \sum_{s \in \Omega_k} (s - \mu_k)^2}. \quad (2.45)$$

Several methods are available to define Ω_k . For example, one can define Ω_k as features in the same layer [Ba et al., 2016], or as features along the same dimension over different samples or input positions [Ioffe and Szegedy, 2015; Ulyanov et al., 2016], or as a hybrid grouping of these dimensions [Wu and He, 2018].

An advantage of normalization is to put features on the same scale and make them comparable. This has been found to be very helpful for stabilizing the training process and improving the convergence behavior. As we will see in subsequent chapters, normalization plays an important role in many successful systems.

As an aside, while the term *normalization* in deep learning usually refers to a process of subtracting the mean and dividing by the standard deviation, it is in fact a **standardization** process. In other areas, by contrast, normalization is more often referred to as a technique that scales all entries of a vector to the interval $[0, 1]$. Standardization has no such requirement. It instead tends to have the input centered around zero. In this sense, the term *normalization* is somewhat of a misnomer in deep learning. Nevertheless, normalization and standardization are used interchangeably in this book when referring to processes like Eq. (2.43).

2.3.5 Residual Units

The success of deep neural networks has been mostly attributed to the increasing number of layers used in forming more complex functions. Although stacking a large number of layers is the simplest way to obtain a deep model, it has been pointed out that such models are difficult to train. There are several reasons for this, such as the vanishing or exploding gradient problem. Furthermore, sequentially feeding a single representation through every layer can be disadvantageous when upper-level feature extractors could actually benefit from direct access to intermediate representations from earlier layers.

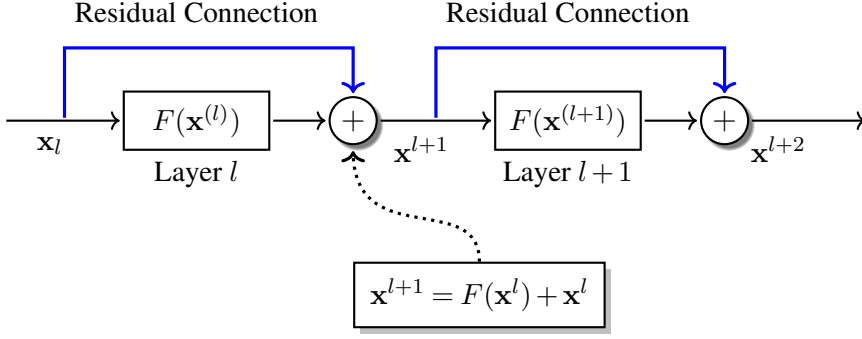


Figure 2.10: Illustration of a 2-layer residual neural network. For each layer (or block), there is a skip or shortcut connection (in blue) that directly adds the input to the output.

Residual neural networks are one of the most effective approaches to addressing these issues [He et al., 2016]. They are a special type of neural network that adds **residual connections** (or **skip connections**, or **shortcut connections**) across layers in a layer stack. Let $F(\mathbf{x})$ be a layer or a sequence of layers that maps $\mathbf{x}$ to some output. A residual block built on $F(\mathbf{x})$ is defined by summing the mapping $F(\mathbf{x})$ and the identity mapping $\mathbf{x}$:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}. \quad (2.46)$$

A more common use of residual connections is in a neural network consisting of a number of identical layers. Let $\mathbf{x}^l$ and $\mathbf{y}^l$ be the input and output of layer l in a residual multi-layer neural network. The output of layer l can be defined as:

$$\mathbf{x}^{l+1} = F(\mathbf{x}^l) + \mathbf{x}^l, \quad (2.47)$$

or

$$\mathbf{y}^l = F(\mathbf{y}^{l-1}) + \mathbf{y}^{l-1}. \quad (2.48)$$

Figure 2.10 shows the architecture of a 2-layer residual neural network. Clearly, the residual connections add the inputs of a layer (or block) directly to its outputs. The added identity mapping is generally thought of as one of the most effective ways to simplify the optimization and ease the information flow in a deep model.

2.4 Training Neural Networks

In this section, we turn to the training problem, which is fundamental in developing neural network-based systems. Most of the discussion here is focused on methods in a supervised learning setting. We will discuss unsupervised methods in Section 2.6.

2.4.1 Gradient Descent

The gradient method has been proven to be one of the most successful methods for training neural networks. The basic idea is to iteratively update parameters so that we can minimize a differentiable loss function. In an update, the values of the parameters are adjusted in a way that the loss decreases the fastest. In a mathematical sense, it requires the update to be in the opposite direction of the gradient of the loss. This is known as **gradient descent** or **steepest descent**. Let θ_t be the parameters at step t (call it an **update step** or a **training step**), and $\mathcal{L}(\theta_t)$ be the loss computed by the model parameterized by θ_t . The **update rule** (or **delta rule**) of gradient descent is given by the equation:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \quad (2.49)$$

where $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ is the gradient of the loss with respect to the parameters at step t . It can be obtained by running the backpropagation algorithm presented in Section 2.1.3. Since θ_t is usually of multiple dimensions, $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ could be a vector or matrix that has the same shape as θ_t . η is the **learning rate** that controls how big a step we take in the direction of the minimum. While η can be simply set to a constant value during training, it is more common to adjust its value as the training proceeds (see Section 2.4.4 for a discussion). See Figure 2.11 for an illustration of gradient descent.

Eq. (2.49) gives a very basic definition of gradient descent. There are a number of improvements to the form of Eq. (2.49). Some of them are:

- **Gradient Descent with Momentum.** In physics, **momentum** is a vector quantity that describes the mass in motion. If we think of updating parameters as moving an object in a space, then we need to consider the momentum of the object at a position to determine the direction of the next move. This idea can be implemented by re-defining the update rule as:

$$\theta_{t+1} = \theta_t + v_t, \quad (2.50)$$

where v_t is the velocity vector of the momentum. In the classic momentum method [Polyak, 1964], v_t is defined to be:

$$v_t = \lambda \cdot v_{t-1} - \eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \quad (2.51)$$

v_t retains some of the previous momentum (i.e., v_{t-1}), followed by a correction based on the gradient (i.e., $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$). λ is a scalar for weighting v_t in an update. A well-known improvement to Eq. (2.51) is to take into account the momentum in the gradient, and thus prevent the velocity from becoming excessively large when approaching the minimum [Nesterov, 1983], like this

$$v_t = \lambda \cdot v_{t-1} - \eta \cdot \frac{\partial \mathcal{L}(\theta_t + \lambda \cdot v_{t-1})}{\partial \theta_t}. \quad (2.52)$$

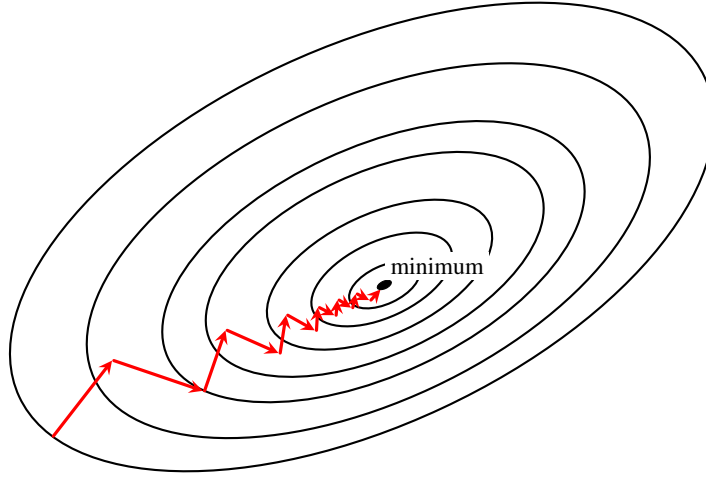


Figure 2.11: Illustration of gradient descent in a 2D space (red lines stand for level sets). The goal is to find the parameters (i.e., values along the two dimensions) that minimize the value of a given loss function. Gradient descent does this by starting at a random point and stepping to the minimum in a number of updates of the parameters. In each update, it adjusts the parameters θ_t in the direction that makes the loss lower. The idea here is that the update chooses the direction of the steepest ascent, that is, the model moves a step in the direction of the negative gradient of the loss (i.e., $-\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$). The size of the move is controlled by a hyper-parameter η , called the learning rate. Thus, the amount of the change to the parameters is $-\eta \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$. By adding this to θ_t , we obtain the new parameters θ_{t+1} . This process repeats for a number of updates until the value of the loss function is close to the minimum.

A more detailed discussion on the difference between Eq. (2.51) and Eq. (2.52) can be found in [Sutskever et al. \[2013\]](#)'s paper.

- **Adaptive Gradient Descent.** In adaptive methods for gradient descent, the update rule is adapted to every parameter, rather than the whole model. **AdaGrad** is a method of this kind [[Duchi et al., 2011](#)]. It scales down the learning rate for frequently updated parameters and scales it up for infrequently updated ones. Assume that θ_t and $\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}$ are both d -dimensional vectors. We can define a new variable $G \in \mathbb{R}^{d \times d}$ as the sum of the

outer product of the gradient over the past t steps⁵:

$$G_t = \sum_{i=1}^t \left[\frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i} \right]^\top \cdot \frac{\partial \mathcal{L}(\theta_i)}{\partial \theta_i}. \quad (2.54)$$

In general, $(G_t)^{\frac{1}{2}} \in \mathbb{R}^{d \times d}$ can be viewed as an indicator that describes to what extent a parameter has been updated so far. However, computing $(G_t)^{\frac{1}{2}}$ is extremely expensive. So it is more common to use the diagonal of G_t instead. Then, the update rule of AdaGrad is given by:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\text{diag}(G_t) + \epsilon}} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \quad (2.55)$$

where $\text{diag}(G_t)$ is the diagonal of G_t , i.e., $\text{diag}(G_t)(k) = G_t(k, k)$. ϵ is a smoothing factor for numerical stability. Instead of summing over those squared gradients in an unweighted manner, another way is to reduce the impact of “old” gradients and make “recent” gradients more important. **AdaDelta** considers this by accumulating squared gradients with a **decay factor** [Zeiler, 2012]:

$$g_t^2 = \sigma \cdot g_{t-1}^2 + (1 - \sigma) \cdot \left(\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \right), \quad (2.56)$$

where σ is the decay factor of a value < 1 . Like Eq. (2.55), the update rule for AdaDelta can be given by replacing $\text{diag}(G_t)$ with g_t^2 :

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{g_t^2 + \epsilon}} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}. \quad (2.57)$$

Since $\sqrt{g_t^2 + \epsilon}$ can be seen as the root mean square (RMS) of the gradient, Eqs. (2.56-2.57) are also known as the **RMSPprop** method [Hinton, 2018].

- **Adam (Adaptive Moment Estimation)**. The Adam optimizer combines the merits of both the adaptive gradient descent and momentum methods [Kingma and Ba, 2014]. It defines an estimate of the mean of the gradient (the first moment) and an estimate of the uncentered variance of the gradient (the second raw moment). Let m_t and v_t be the two

⁵Given two vectors $\mathbf{a} = [a_1 \ \cdots \ a_d]$ and $\mathbf{b} = [b_1 \ \cdots \ b_d]$, the outer product of $\mathbf{a}$ and $\mathbf{b}$ is:

$$\begin{aligned} \mathbf{a} \otimes \mathbf{b} &= \mathbf{a}^\top \cdot \mathbf{b} \\ &= \begin{bmatrix} a_1 \\ \vdots \\ a_d \end{bmatrix} \cdot [b_1 \ \cdots \ b_d] \\ &= \begin{bmatrix} a_1 b_1 & \cdots & a_1 b_d \\ \vdots & \ddots & \vdots \\ a_d b_1 & \cdots & a_d b_d \end{bmatrix}. \end{aligned} \quad (2.53)$$

moment estimates. They are given by the equations:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t}, \quad (2.58)$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot \left(\frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \odot \frac{\partial \mathcal{L}(\theta_t)}{\partial \theta_t} \right), \quad (2.59)$$

where $\beta_1, \beta_2 \in [0, 1]$ are hyper-parameters for a trade-off between the previous estimate and the gradient (or squared gradient) at the current step. β_1 and β_2 are also treated as the decay factors of these averages. For example, common choices for β_1 and β_2 are 0.9 and 0.999. As the initial moments are set to 0, these estimates are biased toward zero vectors at the very beginning of the training process. To address this issue, bias corrections are used in Adam, leading to bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad (2.60)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (2.61)$$

Since $\beta_1, \beta_2 < 1$, the corrections would be sufficiently small if a larger number of updates are performed. The update rule is finally defined to be:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \quad (2.62)$$

Eq. (2.62) resembles the general form of gradient descent, but makes use of both the momentum method (i.e., the moving average of the past gradients) and the adaptive method (i.e., the moving average of the past squared gradients). In practice, Adam has become a popular optimizer for training neural networks.

Improving gradient descent is an active sub-field of deep learning, but a full discussion of all those techniques is beyond the scope of this book. A few related issues will be discussed in the remainder of this section.

On a last note of this subsection, a practical issue that one should consider in utilizing iterative training methods is when to stop training. **Stopping criterion** is a general topic in optimization. For gradient descent and its variants, it is common practice to set a maximum number of training steps or **training epochs**⁶, say 20,000 steps, or 100 epochs. As an alternative, we can perform training until convergence. For example, we can say that the training converges if the loss tends to be stable for a number of training steps. When there is some data for validating the model, a better method may be to check the states of the model on validation data. For example, we can stop the training when the prediction error increases on the validation data. This method, known as **early stopping**, is often used as a means of regularization. In Section 2.5.3, we will see more details about how to apply early stopping during training by using a validation dataset. On the algorithmic side, there has been much

⁶A training epoch means that the trainer goes over the whole training dataset exactly once.

interest and work in studying the convergence and error bounds for machine learning methods. We refer the interested reader to a few textbooks for further discussions [Mohri et al., 2018; Kochenderfer and Wheeler, 2019].

2.4.2 Batching

The loss function is an essential aspect of the training of neural networks. While a number of mathematical forms are available to define the loss function (see Chapter 1), we still need to decide the scale of samples over which to compute the loss function. Perhaps the simplest method is **stochastic gradient descent (SGD)**. In each update of parameters, SGD computes the loss function on a single sample that is randomly selected from the training dataset. Let $\mathcal{D}$ be a set of training samples, and $(\mathbf{x}^{(i)}, \mathbf{y}_{\text{gold}}^{(i)})$ be a randomly selected sample from $\mathcal{D}$. Given a neural network

$$\mathbf{y}_{\theta}^{(i)} = F_{\theta}(\mathbf{x}^{(i)}), \quad (2.63)$$

the loss of SGD is defined to be:

$$\mathcal{L}(\theta) = \mathcal{L}(\mathbf{y}_{\theta}^{(i)}, \mathbf{y}_{\text{gold}}^{(i)}), \quad (2.64)$$

where $\mathcal{L}(\mathbf{y}_{\theta}^{(i)}, \mathbf{y}_{\text{gold}}^{(i)})$ is a sample-level loss function that quantifies the error in the model output $\mathbf{y}_{\theta}^{(i)}$ with respect to the ground truth $\mathbf{y}_{\text{gold}}^{(i)}$.

SGD has been one of the most important optimization methods in machine learning due to its simplicity. However, SGD exhibits high variance in its updates because a single sample provides only a noisy approximation of the true gradient. To estimate the gradient in a more precise way, we can take into account a set of samples (call it a **batch**) in computing the loss. This method is known as **batching**. Let S be a set of samples from $\mathcal{D}$. The loss function is then defined on S , as follows

$$\begin{aligned} \mathcal{L}(\theta) &= \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}_{\text{gold}}) \in S} \mathcal{L}(F_{\theta}(\mathbf{x}), \mathbf{y}_{\text{gold}}) \\ &= \frac{1}{|S|} \sum_{(\mathbf{x}, \mathbf{y}_{\text{gold}}) \in S} \mathcal{L}(\mathbf{y}_{\theta}, \mathbf{y}_{\text{gold}}). \end{aligned} \quad (2.65)$$

If $S = \mathcal{D}$, then we have the **batch gradient descent (BGD)** method, i.e., the gradient is estimated on the entire set of training samples. In general, batch gradient descent is what we would ordinarily call gradient descent. However, calculating the loss on all the training samples simultaneously is time consuming. In practice, it is more common to use a batch much smaller than $\mathcal{D}$. This is known as **mini-batch gradient descent**. It is adopted in learning real-world systems for its good efficiency and strong performance.

As another “bonus”, batching is generally used as a way to make dense computation on matrices for system speed-up. Assume that S consists of m samples $\{(\mathbf{x}^{(i_1)}, \mathbf{y}_{\text{gold}}^{(i_1)}), \dots, (\mathbf{x}^{(i_m)}, \mathbf{y}_{\text{gold}}^{(i_m)})\}$.

We can batch all input vectors and ground-truth vectors as matrices:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{(i_1)} \\ \vdots \\ \mathbf{x}^{(i_m)} \end{bmatrix}, \quad (2.66)$$

$$\mathbf{Y}_{\text{gold}} = \begin{bmatrix} \mathbf{y}_{\text{gold}}^{(i_1)} \\ \vdots \\ \mathbf{y}_{\text{gold}}^{(i_m)} \end{bmatrix}. \quad (2.67)$$

Then, we can run the neural network on the batched input and output, like this:

$$\mathbf{Y}_\theta = F_\theta(\mathbf{X}). \quad (2.68)$$

Likewise, we can compute the batched loss

$$\mathcal{L}(\theta) = \frac{1}{m} \cdot \mathcal{L}(\mathbf{Y}_\theta, \mathbf{Y}_{\text{gold}}), \quad (2.69)$$

where $\mathcal{L}(\mathbf{Y}_\theta, \mathbf{Y}_{\text{gold}})$ vectorizes the computation of $\sum_{k=1}^m \mathcal{L}(\mathbf{y}_\theta^{(i_k)}, \mathbf{y}_{\text{gold}}^{(i_k)})$. Eqs. (2.68-2.69) prevent the repetitive calls of the forward and backward passes on individual samples. They instead pack everything in a single pass through the network. This makes better use of maximum available compute on modern GPUs which are the majority of the devices for running deep learning systems.

2.4.3 Parameter Initialization

Gradient descent requires the training process to start from some initial parameters. Since the training objective in a practical system is often a highly non-convex function with many local minima, the performance of the resulting model is extremely sensitive to the parameter initialization step. Here, we describe some of the most common methods for parameter initialization.

- **Constant Initialization.** The first method could assign the same value to all parameters (or all entries of a parameter matrix). This method, though straightforward, causes all hidden units to update identically during training. As a result, the network becomes unable to learn complex patterns. It generally performs poorly in practice unless specific architectural randomness is introduced.
- **Initialization with Predefined Distributions.** A more practical approach is to randomly initialize parameters using predefined probability distributions. The simplest method of this kind is to draw parameter values from a uniform or Gaussian distribution, e.g., assigning a random value in the interval $[-0.1, 0.1]$. Interestingly, this heuristic is satisfactory for many basic practical applications.
- **Layer-sensitive Initialization.** An extension to basic random initialization is to use tailored distributions based on the dimensions of different neural network layers. Given

a layer $\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$, let d_{in} and d_{out} denote the fan-in and fan-out (i.e., the row and column dimensions of $\mathbf{W}$), respectively. The **LeCun initialization** method [LeCun et al., 2012] assigns a random number to every parameter of $\mathbf{W}$ by considering only the input dimension:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\frac{1}{\sqrt{d_{\text{in}}}}, \frac{1}{\sqrt{d_{\text{in}}}}\right), \quad (2.70)$$

where $U(-a, a)$ denotes a uniform distribution over the interval $[-a, a]$. Likewise, the bias term can be initialized in a similar manner. Building upon this, the standard **Xavier initialization** (also known as Glorot initialization) considers both d_{in} and d_{out} to stabilize the variance of activations and gradients across layers [Glorot and Bengio, 2010]:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right). \quad (2.71)$$

Note that these uniform distributions can be readily replaced by normal distributions with a mean of 0 and a variance of $\frac{1}{d_{\text{in}}}$ (for LeCun initialization) or $\frac{2}{d_{\text{in}} + d_{\text{out}}}$ (for Xavier initialization).

Many parameter initialization methods are designed for certain types of neural networks. For example, Xavier initialization is derived assuming the use of symmetric activation functions, such as the Sigmoid and hyperbolic tangent. For ReLU, one can refer to He et al. [2015]’s work. Another example is initialization for deep neural networks. It has been found that appropriate initialization is critical to the success of extremely deep models in NLP. Considering the model depth as an additional factor in initialization, we can modify Eq. (2.71) to be:

$$\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}} \sim U\left(-\frac{\alpha_s}{l} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}, \frac{\alpha_s}{l} \cdot \sqrt{\frac{6}{d_{\text{in}} + d_{\text{out}}}}\right), \quad (2.72)$$

where l is the depth of a layer, and α_s is a hyper-parameter. Apart from this, several methods have been proposed to address the initialization of deep neural networks, including the Lipschitz initialization [Xu et al., 2020], the T-Fixup initialization [Huang et al., 2020], the Admin initialization [Liu et al., 2020], and so on.

As a final note, in practice, we do not have to restrict training to a single starting point. It is common to perform multiple trials using different initialization methods or random seeds, and then select the best-performing model from these trials. This strategy is particularly helpful when local minima abound in the optimization landscape.

2.4.4 Learning Rate Scheduling

To achieve desirable results, it is essential to carefully configure the learning rate throughout the training process. While some of the update rules, as noted above, have considered scaling the gradient for different parameters, **learning rate scheduling** is conventionally focused more on designing heuristics to adjust η over training steps. In practice, an excessively large learning

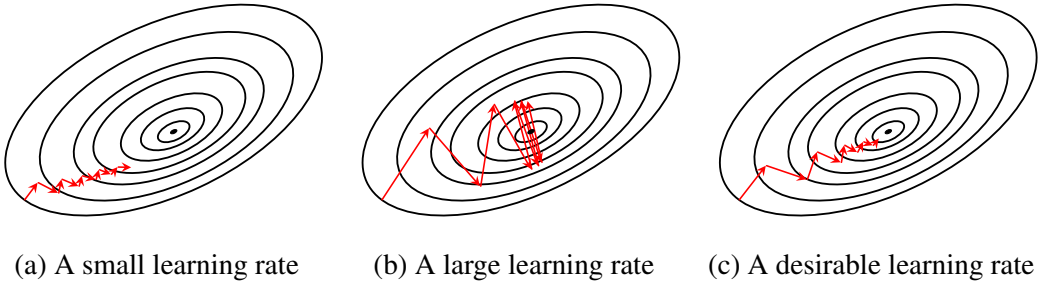


Figure 2.12: Illustrations of learning with different learning rates. Small learning rates (left) help us step toward the minimum in a precise manner, but require significantly more time for convergence. Large learning rates (middle) lead to rapid initial updates, which is highly beneficial when far from the minimum. However, as we approach the minimum, excessively large learning rates cause overshooting. A more desirable strategy (right) is to update the model rapidly when far from convergence, and to slow down the updates as convergence approaches.

rate usually leads to overshooting around the minimum, whereas a learning rate that is too small results in painfully slow convergence (see Figure 2.12). A common strategy is to take large steps at the beginning (i.e., using a large learning rate) and smaller steps as the loss approaches the minimum. Here, we present some of the popular methods for learning rate scheduling.

- **Fixed Learning Rates.** Fixing the learning rate is generally a sub-optimal strategy, but could be used when prototyping systems, e.g., conducting a quick test of a new method by training it for only a few epochs.
- **Learning Rate Decay.** Decay is a commonly used technique for learning rate scheduling. There are many approaches to this idea. For example, one can halve the learning rate after each training epoch. Here we use n_t to represent the current training step, and n_{decay} to represent how often we change the learning rate (e.g., 100 steps). Table 2.2 shows several decay functions for learning rate scheduling.
- **Warmup and Decay.** As noted previously, it is common to initialize model parameters with random values before training a neural network. However, starting from scratch with a large learning rate is usually inadvisable because gradients at the early stages of training are highly noisy, and the state of the model is unstable. Therefore, it is more reasonable to start with a small learning rate and gradually increase it. Then, after the model has trained for some time, the learning rate begins to decay as usual. This intuition motivates the warmup and decay method. A popular formulation of this method was proposed by Vaswani et al. [2017], as follows:

$$\eta_{n_t} = \eta_0 \cdot \min \left(\left(\frac{n_t}{n_{\text{decay}}} \right)^{-0.5}, \frac{n_t}{n_{\text{decay}}} \cdot (n_{\text{warmup}})^{-1.5} \right), \quad (2.73)$$

where η_0 is the initial learning rate, and n_{warmup} is a hyper-parameter that specifies the

Entry	Formula	Hyper-parameters
Piecewise Constant Decay	$\eta_{n_t} = \beta_i$ if $\gamma_i \leq \frac{n_t}{n_{\text{decay}}} < \gamma_{i+1}$	values $\{\beta_1, \dots, \beta_m\}$ thresholds $\{\gamma_1, \dots, \gamma_m\}$
Exponential Decay	$\eta_{n_t} = \eta_0 \cdot \lambda^{\frac{n_t}{n_{\text{decay}}}}$	decay rate λ , init. rate η_0
(Drop) Exponential Decay	$\eta_{n_t} = \eta_0 \cdot \lambda^{\lfloor \frac{n_t}{n_{\text{decay}}} \rfloor}$	decay rate λ , init. rate η_0
Natural Exponential Decay	$\eta_{n_t} = \eta_0 \cdot \exp(-\lambda \cdot \frac{n_t}{n_{\text{decay}}})$	decay rate λ , init. rate η_0
Inverse Time Decay	$\eta_{n_t} = \eta_0 \cdot \frac{1}{1 + \lambda \cdot \frac{n_t}{n_{\text{decay}}}}$	decay rate λ , init. rate η_0
(Drop) Inverse Time Decay	$\eta_{n_t} = \eta_0 \cdot \frac{1}{1 + \lambda \cdot \lfloor \frac{n_t}{n_{\text{decay}}} \rfloor}$	decay rate λ
Cosine Decay	$\eta_{n_t} = \eta_0 \cdot ((1 - \alpha) \cdot c_{\text{decay}} + \alpha)$ $c_{\text{decay}} = \frac{1}{2} \cdot (1 + \cos(\pi \cdot \frac{n_t}{n_{\text{decay}}}))$	coefficient α init. rate η_0

Table 2.2: Decay functions. λ = decay rate, η_0 = initial learning rate, and $\{\beta_i\}$, $\{\gamma_i\}$ and α = other hyper-parameters.

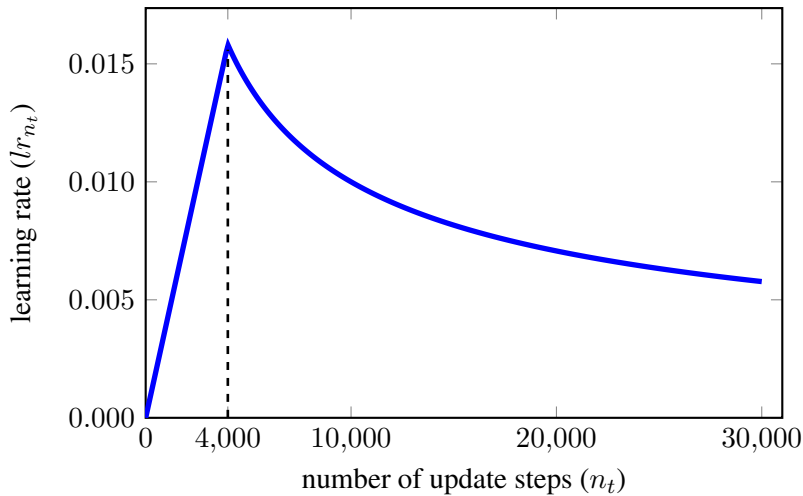


Figure 2.13: Learning rate scheduling: warmup and then decay ($n_{\text{warmup}} = 4,000$, $n_{\text{decay}} = 1$, and $\eta_0 = 1$). The learning rate increases linearly with n_t for the first 4,000 steps. Then, the learning rate follows an inverse square root function and decays as the training continues. The change in the learning rate will be small if n_t is sufficiently large, indicating the fine adjustment of the parameters when we are approaching the minimum of the loss.

number of steps for the warmup phase. Figure 2.13 plots the curve of Eq. (2.73) where n_{warmup} , n_{decay} , and η_0 are set to 4,000, 1 and 1, respectively. We see that the learning rate increases linearly in the first n_{warmup} steps and then decays as an inverse square root function.

Choosing an appropriate learning rate scheduling strategy is a highly empirical problem, and there are no universally optimal choices. The problem becomes even more complex when considering the correlation between the learning rate and other training dynamics, even though

learning rate scheduling is typically treated as an isolated task. For example, when a larger batch size is used in training, a larger learning rate is generally required to achieve good results [Ott et al., 2018; Smith et al., 2018]. As a result, selecting an optimal learning rate remains difficult and time-consuming in neural network applications. Occasionally, one must rely on numerous trial-and-error runs to find a desirable learning rate setup for the specific problem at hand.

2.5 Regularization Methods

We now discuss the regularization methods for preventing overfitting. While regularization is a wide-ranging topic in machine learning, we present some of those that are commonly adopted in training neural networks.

2.5.1 Norm-based Penalties

One of the most popular methods involves a regularization term based on the l_p norm. A general form of the regularized objective can be defined as:

$$\hat{\theta} = \arg \min_{\theta} \mathcal{L}(\theta) + \alpha \cdot R(\theta), \quad (2.74)$$

where $R(\theta)$ is the regularization term weighted by a coefficient α . In general, $R(\theta)$ serves as an additional loss that penalizes complex models. This is motivated by the fact that complex models are more likely to overfit the data (see Chapter 1). To penalize model complexity, a straightforward approach is to define $R(\theta)$ as the l_1 norm of the parameter vector θ . The l_1 norm-based penalty is given by

$$R(\theta) = \sum_i |\theta_i|. \quad (2.75)$$

Eq. (2.75) penalizes models with very large parameter values. This can be intuitively understood by analogy to a polynomial function: large coefficients of variables in a polynomial function lead to a complex curve. Typically, regularization with the l_1 norm is referred to as l_1 regularization or **Lasso regularization**. This method requires no fundamental modifications to the optimization algorithm and can be implemented using standard gradient descent. More interestingly, l_1 regularization typically provides sparse solutions to the original training objective. It forces many parameters to be exactly zero, leaving only a small subset of critical features with non-zero weights. This also implies an inherent ability of feature selection because parameters are forced to be zero for less important features.

An alternative to l_1 regularization is l_2 **regularization**, also widely known as **Ridge regularization**. In l_2 regularization, the regularization term is given by the squared l_2 norm of the parameters

$$R(\theta) = \sum_i \theta_i^2. \quad (2.76)$$

Like the l_1 norm, the l_2 norm penalizes parameter vectors that deviate significantly from the origin. However, it slightly differs from the l_1 norm in that the l_2 norm enforces all parameters to have small values (but not necessarily to be zero) and there are no large value parameters. In this sense, the use of the l_2 norm does not introduce strict sparsity into the solution, but rather performs a “smoothing” effect on the underlying distribution of features. Hence, it is sometimes called **weight decay** to emphasize its ability to continuously shrink parameter values during gradient updates.

In a broader sense of machine learning, Eq. (2.74) offers a generalized framework for injecting prior knowledge into the training of a neural network. There are numerous ways to design the regularization term, and mitigating overfitting is merely one objective of these designs. We will encounter various applications of this approach in NLP, and we will explore a few examples in the remaining chapters of this book.

2.5.2 Dropout

In a real-world neural network, a layer typically involves hundreds or thousands of neurons and produces a feature vector accordingly. While each of these features is computed by a single neuron, they work together to form the input to each neuron of the following layer. As a result, a feature is forced to cooperate with other features. It is like a group of people sitting together and making a collective decision. Although a member could have opinions independently, they occasionally try to correct errors made by the rest of the group. In this case, every group member is co-adapted to others in the group [Hinton et al., 2012]. From a feature engineering standpoint, the **co-adaptation** of neurons helps when modeling complex problems, as it implicitly makes some sort of higher order features. Beyond this, the strong supervision information (e.g., propagating errors through layers) could strengthen the co-adaptation in training. This explains more or less why a neural network with a large number of neurons can fit complex curves. At test time, however, the co-adaptation prevents generalization. Since all neurons of a layer are learned to collaborate well on the training data, a small change in the input could affect all these neurons and lead to a big change in the behavior of the neural network.

A way to mitigate or eliminate complex co-adaptations is to learn for each neuron to predict in the absence of other neurons. To this end, one can simply drop some of the neurons in training. This method is known as **dropout** [Srivastava et al., 2014]. Let n be the number of neurons of a layer. Given a probability ρ (call $1 - \rho$ the **dropout rate**), we can generate an n -dimensional mask vector $\mathbf{M}_{\text{drop}}$ where every entry is set to 1 with a probability of ρ , and set to 0 with a probability of $1 - \rho$. Then, a dropout layer can be defined as

$$\mathbf{y} = \mathbf{M}_{\text{drop}} \odot \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B}), \quad (2.77)$$

where $\psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$ is a usual single-layer neural network. Eq. (2.77) only activates the neurons whose masks are 1. For dropped neurons, all connections from/to these neurons are blocked (see Figure 2.14 (a)). During training, $\mathbf{M}_{\text{drop}}$ is randomly generated in a call of the forward and backward passes. A neuron therefore can learn to work with different

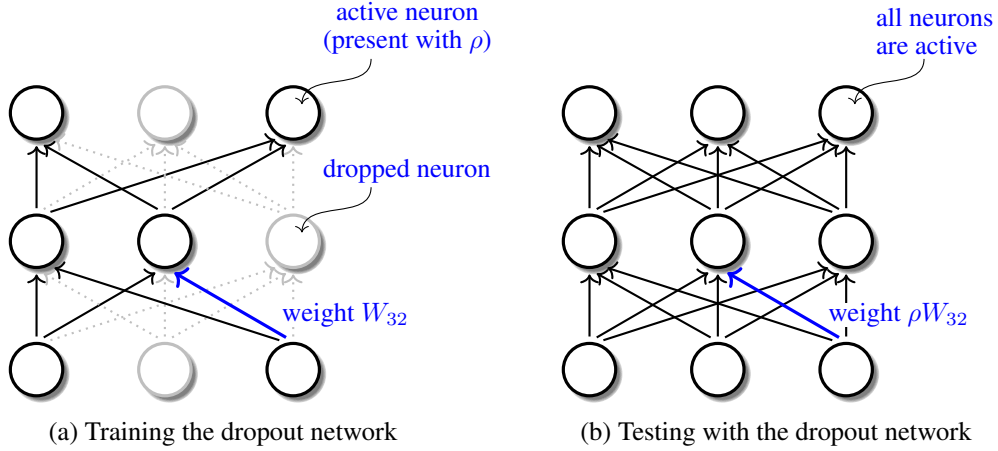


Figure 2.14: Illustration of dropout for a multi-layer neural network (training vs test). At training time, every neuron is randomly dropped with a probability of $1 - \rho$, resulting in a slimmed network. In this sense, dropout training is essentially a process of learning an exponentially large number of sub-networks. At test time, the full network is used as usual, which is the result of combining all those sub-networks for prediction. Since all connections between neurons are activated with the probability ρ during training, the weights of the inference network are scaled down with ρ .

neurons each time and would not adapt to the same group of “co-workers”. Another way to understand dropout is to view it as learning sub-models of a “big” model. The use of Eq. (2.77) is essentially a sampling process that extracts a sub-network from the original network. So, training with dropout is akin to training an exponentially large ensemble of sub-networks⁷. On the other hand, the training is efficient because these sub-networks share the same parameters for the same neuron and the update of a parameter can benefit exponentially many sub-networks.

At test time, all these sub-networks are combined for prediction. In this case, we do not need to drop any neuron but use the original network as usual. This makes it simple to implement dropout: a neuron is present with some probability on the training data, and all neurons are present and work together on the test data. Since the connections between neurons are involved with a probability of ρ in training, the learned weights are scaled down with ρ in the inference network, i.e., a layer has a form:

$$\mathbf{y} = \psi(\mathbf{x} \cdot \rho \mathbf{W} + \rho \mathbf{B}), \quad (2.78)$$

where the weights $\mathbf{W}$ and $\mathbf{B}$ are scaled by ρ ⁸.

⁷For a single-layer network having n neurons, there are 2^n possible sub-networks.

⁸Note that placing the scaling factor inside the activation function ψ (e.g., using $\rho \mathbf{W}$) implicitly assumes that ψ is a linear function or possesses positive homogeneity (e.g., the ReLU function, where $\text{ReLU}(cx) = c\text{ReLU}(x)$ for $c > 0$). If a non-linear activation function is used, such as Sigmoid, the scaling factor cannot be distributed inside. In such cases, the scaling is applied directly to the output of the activation function to properly preserve the

See Figure 2.14 for a comparison of training and applying a dropout network. Eq. (2.78) requires an update of the inference system. An alternative is to take into account the scaling issue only in the training process and leave the inference system as it is. For example, we can scale up all the parameters with $\frac{1}{\rho}$ in dropout training, like this

$$\mathbf{y} = \mathbf{M}_{\text{drop}} \odot \psi(\mathbf{x} \cdot \frac{1}{\rho} \mathbf{W} + \frac{1}{\rho} \mathbf{B}). \quad (2.79)$$

Since multiplying $\frac{1}{\rho} \mathbf{W}$ with ρ yields $\mathbf{W}$ (this also holds for the bias term $\mathbf{B}$), we can use $\mathbf{W}$ (and $\mathbf{B}$) as the parameters of the inference system.

2.5.3 Early Stopping

In Chapter 1 we have discussed a bit of how to stop the training by monitoring the performance on the validation data. This can be viewed as a form of model selection that seeks an optimal balance between underfitting and overfitting. Note that early stopping is not merely an empirical heuristic; it is also well-founded in statistical learning theory. For example, researchers have found that, under certain conditions, early stopping has an effect similar to l_2 regularization, restricting the learning to a region of small parameter values [Bishop, 1995a; Goodfellow et al., 2016]. Furthermore, studies show that specific early stopping rules share a close relationship with the bias-variance trade-off and can guarantee desirable convergence properties [Yao et al., 2007].

In practice, however, early stopping requires several heuristics to make it practical and useful. The primary problem lies in determining the exact stopping criterion. Ideally, one might imagine that there is a perfect U-shaped error curve on the validation data, and the training can be halted immediately when the error starts to increase. The truth, however, is that the error curve cannot be simply described as a strictly convex function of the training time. After drops in the error in a certain number of training steps, the performance of the model tends to fluctuate, leading to many local minima. The problem would be more interesting if one wants to save time and stop the training as early as possible. However, we never know whether the current choice or decision is the best one because we have no idea of what happens next. A commonly used method is to decide whether the training should stop by checking the model states for a number of past update steps (or epochs) [Prechelt, 1998]. Some early stopping conditions are:

- The change in the performance is below a threshold for a given number of steps (or epochs).
- The change in the model parameters is below a threshold for a given number of steps (or epochs).
- The average performance over a given number of steps (or epochs) starts to decrease.
- The maximum performance over a given number of steps (or epochs) starts to degrade.

expected value. The mathematical forms for standard inference scaling and inverted dropout training would then be $\mathbf{y} = \rho \cdot \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$ and $\mathbf{y} = \frac{1}{\rho} (\mathbf{M}_{\text{drop}} \odot \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B}))$, respectively.

However, using the model at the point that we stop the training is not always a good choice. In practice, a model often has a large variance in generalization error around that point, making model selection more difficult. Instead of “selecting” a model, an alternative way is to combine multiple models. For example, we can save the model for every run of a given number of training steps (call each copy of the model a checkpoint). The final model is obtained by averaging the parameters of the last few checkpoints. For better results, one may use more sophisticated ensemble methods (see Chapter 1).

2.5.4 Smoothing Output Probabilities

In statistics, smoothing refers to the process of reducing the influence of noisy data points (typically extreme values) and reallocating probability mass to normal data points or unobserved events. It is widely used when a distribution is estimated from limited data, where the probabilities of rare events are poorly represented. For example, consider the language modeling problem described in Section 2.2. A language model is trained in a way that enforces the model to output a one-hot distribution, that is, the entire probability mass of 1 is assigned to a single word, leaving other words assigned zero probabilities. It may be more desirable to distribute the probability to all words, even though many of them are not observed to be the answer given the previous words. In this way, the model learns to make a soft prediction of word probabilities so that it can generalize better on unseen data.

Given a distribution $\mathbf{p} = [p_1 \ \cdots \ p_n]$, it is the purpose of smoothing that we obtain the new estimate between $\mathbf{p}$ and a uniform distribution $\frac{1}{n}$. A common approach to this idea is to use a **shrinkage estimator** to improve $\mathbf{p}$ by making it closer to $\frac{1}{n}$. For example, the additive smoothing mentioned in Chapter 1 is a simple type of shrinkage estimator. Here we consider, for example, smoothing a multinomial distribution. Let p_k denote the probability of event k and s_k denote a quantity that describes some observed “count” of the event. The probability p_k is given by

$$p_k = \frac{s_k}{\sum_{k=1}^n s_k}. \quad (2.80)$$

Then, the smoothed version of p_k is defined as

$$\hat{p}_k = \frac{s_k + \alpha}{\sum_{k=1}^n (s_k + \alpha)}. \quad (2.81)$$

It simply adds a quantity α to each s_k . The value of α controls the smoothness of the resulting estimate. For example, $\hat{p}_k = p_k$ if $\alpha = 0$, and $\hat{p}_k \approx \frac{1}{n}$ as $\alpha \rightarrow \infty$.

Apart from additive smoothing, we can smooth a distribution using a Softmax function with temperature scaling, defined as follows:

$$\hat{p}_k = \frac{\exp(s_k/\beta)}{\sum_{k=1}^n \exp(s_k/\beta)}. \quad (2.82)$$

This form is a classic instance of the **Boltzmann distribution** [Uffink, 2017], where s_k can

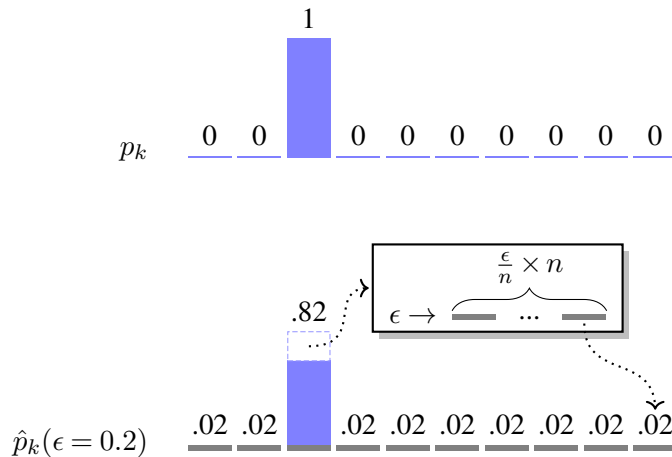


Figure 2.15: Illustration of smoothing a distribution by interpolating it with a uniform distribution: $\hat{p}_k = (1 - \epsilon) \cdot p_k + \frac{\epsilon}{n}$. Mathematically, this operation scales down each original probability p_k by an amount of ϵp_k , pooling a total probability mass of ϵ . This mass is then redistributed evenly across all n dimensions, ensuring every variable receives an additive smoothing factor of $\frac{\epsilon}{n}$.

be viewed as the negative energy of a state, and β represents the temperature that controls the degree of smoothing. Note that s_k can be interpreted in various ways. In a neural network, for example, s_k is typically the pre-activation logit of a neuron. Occasionally, s_k can even be a raw probability. This flexibility implies that we can directly apply Eqs. (2.81–2.82) to any distribution $\mathbf{p}$ even without prior knowledge of how $\mathbf{p}$ was originally estimated. We can rewrite these equations by substituting s_k with p_k :

$$\hat{p}_k = \frac{p_k + \alpha}{\sum_{k=1}^n (p_k + \alpha)}, \quad (2.83)$$

$$\hat{p}_k = \frac{\exp(p_k/\beta)}{\sum_{k=1}^n \exp(p_k/\beta)}. \quad (2.84)$$

Another straightforward method of smoothing is to linearly interpolate $\mathbf{p}$ with a uniform distribution. A standard form of this interpolation is given by:

$$\hat{p}_k = (1 - \epsilon) \cdot p_k + \frac{\epsilon}{n}, \quad (2.85)$$

where ϵ is a hyper-parameter indicating the extent to which we rely on the uniform distribution. To illustrate how Eq. (2.85) works, suppose $\mathbf{p}$ is a one-hot vector (e.g., $p_k = 1$ if $k = z$, and $p_k = 0$ otherwise). By applying Eq. (2.85), we effectively scale down the original distribution by $(1 - \epsilon)$ and add a uniform probability mass of $\frac{\epsilon}{n}$ to every dimension. This redistribution makes the resulting distribution flatter and smoother. See Figure 2.15 for a visual illustration.

In NLP, since many systems make probability-like predictions, a common application of smoothing is to smooth a system's output. There are two ways. First, we can smooth the

ground-truth probability such that the model is guided by the generalized error rather than the error made by hard decisions. For example, the **label smoothing** technique adopts the same form as Eq. (2.85) and improves the ground-truth representation on categorical data [Szegedy et al., 2016]. Second, we can reduce the sharpness and increase the tailedness of a predicted distribution⁹. This method is often used when the posterior probability of the prediction is required, such as minimum Bayesian risk decoding/training [Bickel and Doksum, 2015; Goodman, 1996; Kumar and Byrne, 2004].

2.5.5 Training with Noise

Above, we have shown that adding some amount to each observed count of events in predicting a probability can improve generalization. From a **robust statistics** point of view [Olive, 2022], this is equivalent to improving the robustness of an estimator where a skewed distribution often leads to a biased model. The addition of a small perturbation to the estimate can prevent large biases caused by outliers and unexpected observations of rare events. In this sense, smoothing can be regarded as a way of introducing noise into training, that is, we impose a prior of uniform distribution on the estimate though the correct estimate may not be uniform.

Training with noise is based on the idea that a model should learn to operate under non-ideal conditions, thereby avoiding the overfitting of extreme data points. Here, the term *noise* takes on a broad meaning, and there are several distinct ways to regularize training via noise injection. One of the simplest methods is to introduce noise into the training objectives. For example, label smoothing (as discussed previously) can be viewed as injecting noise into the ground-truth annotations. Alternatively, we can inject random noise directly into the inputs, intermediate hidden states, or outputs of a neural network. A ubiquitous choice is **Gaussian noise**. Suppose we have an activation vector $\mathbf{x} \in \mathbb{R}^n$. The addition of Gaussian noise yields a new corrupted vector as follows:

$$\mathbf{x}_{\text{noise}} = \mathbf{x} + \mathbf{g}, \quad (2.86)$$

where $\mathbf{g} \in \mathbb{R}^n$ is the noise vector. Each element of $\mathbf{g}$ is drawn independently from a Gaussian distribution:

$$\mathbf{g} \sim \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2)). \quad (2.87)$$

For entry k of $\mathbf{g}$, it defines the probability $\Pr(g_k)$ to be:

$$\Pr(g_k) = \frac{1}{\sigma_k \sqrt{2\pi}} \cdot \exp\left(-\frac{(g_k - \mu_k)^2}{2\sigma_k^2}\right), \quad (2.88)$$

where μ_k is the mean, and σ_k is the standard deviation. Typically, μ_k is set to 0. The standard deviation σ_k is a hyper-parameter controlling the magnitude of the injected noise. A larger σ_k means the random noise spreads over a wider region around the mean, increasing the likelihood

⁹In statistics, this is loosely related to transforming a leptokurtic distribution into a more mesokurtic or platykurtic one to prevent overconfidence.

of generating severe perturbations.

Eq. (2.86) is highly generic and can be applied almost anywhere within a neural network. Given a layer $\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B})$, noise (denoted as $\mathbf{g}_{\text{input}}$) can be added directly to the input:

$$\mathbf{y} = \psi((\mathbf{x} + \mathbf{g}_{\text{input}}) \cdot \mathbf{W} + \mathbf{B}). \quad (2.89)$$

Likewise, the noise (say $\mathbf{g}_{\text{output}}$) can be added to the activation (or output):

$$\mathbf{y} = \psi(\mathbf{x} \cdot \mathbf{W} + \mathbf{B}) + \mathbf{g}_{\text{output}}. \quad (2.90)$$

For example, one can simply corrupt the raw inputs (or targets) and run the network as usual, or alternatively, inject random noise into the activations of all hidden layers. While it is common practice to add noise to layer inputs and activations [Plaut et al., 1986; Holmström and Koistinen, 1992; Bishop, 1995b], another distinct approach is to add random noise directly to the model parameters or gradients [Graves et al., 2013; Neelakantan et al., 2015]. For example, perturbing the weight matrix takes the following form:

$$\mathbf{y} = \psi(\mathbf{x} \cdot (\mathbf{W} + \mathbf{g}_w) + \mathbf{B}), \quad (2.91)$$

where $\mathbf{g}_w$ is a noise matrix matching the dimensions of $\mathbf{W}$. Furthermore, we can inject noise (denoted as $\mathbf{g}_{\text{gradient}}$) into the loss gradients with respect to $\mathbf{W}$. Let $\mathbf{s} = \mathbf{x} \cdot \mathbf{W} + \mathbf{B}$ represent the pre-activation logits. The noisy gradient can be written as:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{W}} &= \mathbf{x}^\top \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{s}} + \mathbf{g}_{\text{gradient}} \\ &= \mathbf{x}^\top \cdot \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \frac{\partial \mathbf{y}}{\partial \mathbf{s}} \right) + \mathbf{g}_{\text{gradient}} \\ &= \mathbf{x}^\top \cdot \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \odot \psi'(\mathbf{s}) \right) + \mathbf{g}_{\text{gradient}}. \end{aligned} \quad (2.92)$$

The use of noisy gradients has been found to not only be helpful for robust training but also to ease the gradient flow in the network [Gulcehre et al., 2016].

It should be noted that noise is only present during training and the model works without the addition of noise when making predictions on new data. In this sense, many of the regularization methods could fall under the noisy training framework that is used to prevent fitting the training data precisely and enable the inference system to generalize well on the test data. For example, dropout randomly masks out layer activations so that every neuron learns to work in a noisy environment. At test time, all neurons work together as in a usual neural network.

An additional advantage of noisy training is that the use of random noise creates “new” training samples. Even for the same sample, different noise could lead to different training results. In other words, we essentially train the model on an infinite number of samples. This idea is also linked to another line of research on training with synthetic data, called **data augmentation**. In simple terms, data augmentation encompasses a suite of techniques designed to

generate new training samples from existing ones. An example is **back-translation** [Sennrich et al., 2016]. When developing a machine translation system from language A to language B, we can first train a reverse translation system (say the $B \rightarrow A$ system) on the bilingual data. Then, we use the $B \rightarrow A$ system to translate some additional target-language data to source-language data. This results in new bilingual data where the target-language data is real and the source-language data is synthetic. This new data can be used together with other bilingual data to train the $A \rightarrow B$ system. Beyond back-translation, there are many data augmentation methods in NLP, including replacing words with synonyms, swapping two words, and deleting or inserting words. Furthermore, augmentation can be performed in the continuous latent space, such as substituting a word vector with a geometrically proximate vector. Since data augmentation covers a wide variety of topics, we refer the reader to a few survey papers for further details [Feng et al., 2021; Shorten and Khoshgoftaar, 2019].

One last note on data augmentation: synthetic data can also be crafted for specific objectives. A popular idea is **adversarial machine learning**. It generates **adversarial samples** on which a model would make mistakes (call such processes **attacks**) [Szegedy et al., 2014; Goodfellow et al., 2015]. The model is trained to make correct predictions on these samples, i.e., it defends against the attacks. For example, in some cases, a machine translation system might output completely erroneous translations if the gender of the subject in the input sentence is merely flipped. For a more robust system, one may train the translation model by using more gender-balanced data, gathered either manually or automatically. But it is not easy to craft samples that look like normal sentences but can fool the model [Zhang et al., 2020]. This in turn makes it interesting yet challenging to generate adversarial samples in NLP, since a small change in a sentence (such as word replacement) could lead to something with a very different meaning¹⁰. The challenge also motivates a thread of research on investigating adversarial samples in NLP [Jia and Liang, 2017; Belinkov and Bisk, 2018; Ebrahimi et al., 2018; Alzantot et al., 2018].

2.6 Unsupervised Methods and Auto-encoders

Unsupervised learning is concerned with discovering the underlying patterns in a set of unlabeled data points. A number of problems can be viewed as classical unsupervised learning problems, although we will not discuss them in detail in this chapter. For example, data clustering aims to find groupings in a collection of data objects, without explicit supervision regarding the correct groupings. Another well-known example is association rule mining, which is often framed as a process of establishing relationships among sets of data objects. While these problems are indeed covered by unsupervised learning, we will focus on problems of unsupervised representation learning or **feature learning**, where a model learns to map an object from an input space to a low-dimensional feature vector space¹¹.

¹⁰By contrast, in computer vision, it is much easier to create adversarial samples by making a small change in the input (e.g., pixels), since the input space is continuous and a small input perturbation has very little effect on the whole image.

¹¹In addition to learning to represent data objects, this section also covers some topics on the generation of data objects. We will explore them in Section 2.6.3.

Learning low-dimensional representations has been extensively studied in the context of finding a linear transformation from the original space to a new space. For example, **principal component analysis (PCA)** and its variants aim to find a linear mapping function such that a (high-dimensional) data object can be represented by its coordinates along the directions of greatest variance [Pearson, 1901; Wold et al., 1987]. Here, we extend the mapping function to its natural non-linear generalization and use neural networks as a solution to the mapping problem.

As with other machine learning models, a neural network is typically trained by optimizing model parameters with respect to some loss function. A considerable challenge with unsupervised learning is that there are no ground-truth labels to guide the learning process. One solution to this issue is to resort to non-parametric methods or heuristics (see Chapter 1). However, such methods themselves are not designed to address the optimization issues of large-scale neural networks, particularly when a network is built with a massive number of parameters. In the unsupervised training of neural networks, therefore, it is more common to derive supervisory signals directly from the input data itself. While there are several ways to achieve this [Hopfield, 1982; Ackley et al., 1985; Dayan et al., 1995; Hinton and Salakhutdinov, 2006], we will focus on **auto-encoders** in this section. We choose auto-encoders for discussion because they resemble the general form of supervised models and can be seamlessly trained via back-propagation.

An auto-encoder is a type of neural network that tries to reconstruct the input data from its representation. It is inspired by the idea of dimensionality reduction:

High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors.

– Hinton and Salakhutdinov [2006]

This also develops the idea of representation learning in that the information of an object can be sufficiently represented by a low-dimensional real-valued vector. Typically, an auto-encoder involves a (probably non-linear) dimensionality reduction function (call it an **encoder**) to map the input object to its low-dimensional feature vector representation (call it a **code**). Also, it involves a reverse function (call it a **decoder**) that maps the code back to the object. So, although an auto-encoder is called an “encoder”, it is not just an encoder but a combination of an encoder and a decoder. More formally, let $\mathbf{x} \in \mathbb{R}^{d_x}$ be the input vector of the model, such as a high-dimensional representation of a word. The encoder outputs a vector describing the code or low-dimensional representation of $\mathbf{x}$, denoted as $\mathbf{h} \in \mathbb{R}^{d_h}$. This vector is defined as:

$$\mathbf{h} = \text{Encode}(\mathbf{x}), \quad (2.93)$$

where $\text{Encode}(\cdot)$ is the encoding network. $\text{Encode}(\cdot)$ is typically a multi-layer neural network

and works as a plug-in module for other systems. Thus, $\text{Encode}(\cdot)$ is a general-purpose model. In subsequent chapters, we will see many examples where encoders are trained and applied as parts of “bigger” systems.

Once we obtain the code, we use the decoder to map it back to the input space:

$$\tilde{\mathbf{x}} = \text{Decode}(\mathbf{h}), \quad (2.94)$$

where $\tilde{\mathbf{x}} \in \mathbb{R}^{d_x}$ is the reconstruction of the input, and $\text{Decode}(\cdot)$ is the decoding network. Given the original input $\mathbf{x}$ and the reconstructed input $\tilde{\mathbf{x}}$, the objective of the auto-encoder is to minimize the discrepancy between $\mathbf{x}$ and $\tilde{\mathbf{x}}$. Suppose that the encoder and the decoder are parameterized by θ and ω , denoted as $\text{Encode}_\theta(\cdot)$ and $\text{Decode}_\omega(\cdot)$, respectively. The training objective over a set of samples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ is defined as

$$\begin{aligned} (\hat{\theta}, \hat{\omega}) &= \arg \min_{(\theta, \omega)} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, \tilde{\mathbf{x}}^{(i)}) \\ &= \arg \min_{(\theta, \omega)} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, \text{Decode}_\omega(\mathbf{h}^{(i)})) \\ &= \arg \min_{(\theta, \omega)} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, \text{Decode}_\omega(\text{Encode}_\theta(\mathbf{x}^{(i)}))), \end{aligned} \quad (2.95)$$

where $\mathcal{L}(\cdot)$ is the loss function that computes the discrepancy between $\mathbf{x}$ and $\tilde{\mathbf{x}}$. It is sometimes called the **reconstruction loss**. Popular loss functions for reconstruction include mean squared error loss, crossentropy loss, etc.

Putting together the encoder and the decoder, it is tempting to think of a network in which we feed something into the input layer and get back the same thing out of the output layer. The challenge here is that the low-dimensional vector $\mathbf{h}$ serves as a bottleneck in information flow. There is a risk of information loss in transformation either from $\mathbf{x}$ to $\mathbf{h}$ or from $\mathbf{h}$ to $\tilde{\mathbf{x}}$, making it difficult to “copy” the input to the output. Rather, we need to “squeeze” an object from a high-dimensional space to a dense, low-dimensional space, and then “unsqueeze” it from the new space to the original high-dimensional space. A consequence of this squeeze-and-unsqueeze process is that the encoder is forced to compress the data but retain the information as much as possible. So, the auto-encoder discussed here is also called the **undercomplete auto-encoder**, because $\mathbf{h}$ has a smaller size than $\mathbf{x}$ and $\tilde{\mathbf{x}}$. Figure 2.16 shows an illustration of the undercomplete auto-encoder structure.

Given the loss function $\mathcal{L}(\cdot)$, the encoder $\text{Encode}_\theta(\cdot)$ and the decoder $\text{Decode}_\omega(\cdot)$, the parameters $\hat{\theta}$ and $\hat{\omega}$ can be optimized by using the gradient descent method as in supervised learning (see Section 2.4.1). When applying the auto-encoder, one can simply drop the decoder and use the encoder as a feature extractor, that is, given a new input $\mathbf{x}_{\text{new}}$, we generate a new representation

$$\hat{\mathbf{h}}_{\text{new}} = \text{Encode}_{\hat{\theta}}(\mathbf{x}_{\text{new}}). \quad (2.96)$$

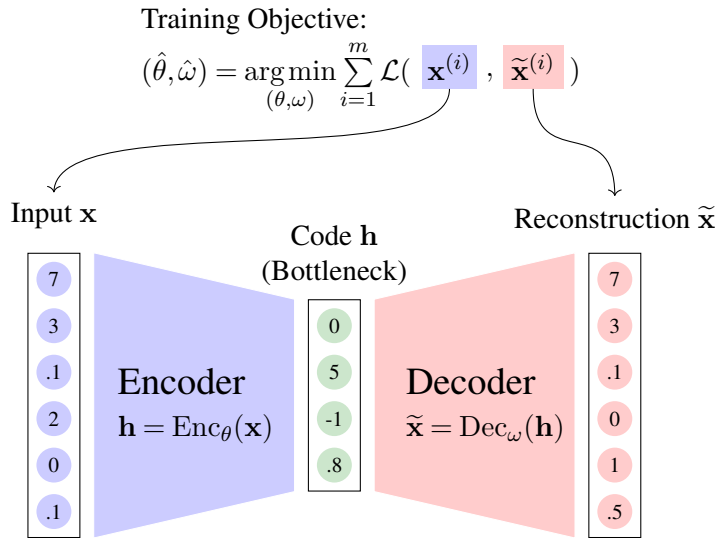


Figure 2.16: Illustration of an undercomplete auto-encoder with an encoder, a decoder and a bottleneck layer sandwiched between them. An input $\mathbf{x}$ (left) is transformed into a code $\mathbf{h}$ (middle) and then a reconstruction $\tilde{\mathbf{x}}$ (right). The parameters of both the encoder and decoder are optimized by minimizing the discrepancy between the input $\mathbf{x}$ and the reconstruction $\tilde{\mathbf{x}}$ on a number of unlabeled samples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$. On new samples, we throw away the decoder, and use the encoder to generate new codes or representations.

Note that the encoder is not a standalone system but typically works with other models for a complete working system. For example, we can train an auto-encoder on some sentences and place a Softmax layer on the output of the encoder to build a sentence classifier. The classifier can be further trained on some task-specific data to solve a new problem, such as tagging a sentence with its sentiment polarity. This also makes the application of auto-encoder fall under the general paradigm of pre-training: a sub-model (i.e., an encoder) is first trained on large-scale, task-agnostic data, and then used as a component of a bigger model on a downstream task.

2.6.1 Auto-encoders with Explicit Regularizers

As more complex neural networks are involved, an auto-encoder tends to learn an identity transformation although the bottleneck makes it a bit harder to pass through without information loss. This is what we would ordinarily expect: we could make $\mathbf{h}$ a surrogate of $\mathbf{x}$ and decode $\mathbf{h}$ to something very similar to $\mathbf{x}$. On the other hand, learning an exact identity transformation requires a highly complicated model and is heavily prone to overfitting. Fortunately, as with other machine learning models, we can regularize the training by using the methods presented in Section 2.5. One of the most popular methods is adding an explicit regularization term to the loss function. Taking together Eq. (2.74) and Eq. (2.95), we can define the training objective to

be:

$$(\hat{\theta}, \hat{\omega}) = \arg \min_{(\theta, \omega)} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, \text{Decode}_{\omega}(\text{Encode}_{\theta}(\mathbf{x}^{(i)}))) + \alpha \cdot R, \quad (2.97)$$

where R is the regularization term accounting for some prior knowledge we want to impose on the training, and α is its weighting coefficient. A common choice for R in auto-encoders is a **sparsity penalty** (resulting in what is known as a **sparse auto-encoder**). The simplest way to implement such a penalty is to apply the l_1 or l_2 norm on the code, as follows:

$$R_{l_1} = \sum_{i=1}^m \sum_k |h_k^{(i)}|, \quad (2.98)$$

$$R_{l_2} = \sum_{i=1}^m \sqrt{\sum_k (h_k^{(i)})^2}. \quad (2.99)$$

It is worth noting that, unlike those penalties on model parameters (see Section 2.5.1), the sparsity penalty regularizes the code $\mathbf{h}$ (or the output of the encoder) to be sparse. The idea of encouraging sparseness in representations stems from **sparse coding** [Olshausen and Field, 1997]. It states that the information of an object is embedded in complex dependencies among the original attributes (or features) of the object. A desirable representation learning system should extract such dependencies and reform them to be a set of independent features. And there should be a small number of these independent features that are active, while the active features vary when we switch to a new object. Note also that, from a Bayesian learning point of view, other penalties in regularized training could be interpreted as priors over models. The sparsity penalty, however, is not a prior because it does not depend on models (or model parameters) but on the training data [Goodfellow et al., 2016]. In this view, the sparsity penalty should not be treated as a “regularization” term, but simply some distribution over the model’s intermediate states. On the other hand, the sparseness of the code, though not well explained by conventional use of regularization terms, is indeed helpful in many applications of auto-encoders, because it directly models the way of representing the input and imposing “priors” on outcomes of encoders. When considered from an empirical point of view, the sparsity penalty is still thought of as a regularizer that biases the training to certain models.

There are other choices for defining the regularization term R in addition to Eqs. (2.98-2.99). For example, a way of forcing sparsity is to penalize the cases where the average value of each entry h_k is far away from a predefined value [Nair and Hinton, 2009]. When h_k chooses values from $[0, 1]$, the regularization can be implemented by defining R as the KL divergence between the average code over a number of samples and the expected code¹². Let $\bar{\mathbf{h}}$ denote the average code over $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$, where the value of $\bar{h}_k$ is the mean of the k -th variable of

¹²In general, we can set all entries of the expected code to $\tau \in [0, 1]$. Sparse codes will be preferred if τ is close to 0, as features are “inactive” in most cases. By contrast, dense codes will be preferred if τ is close to 1.

the code:

$$\bar{h}_k = \frac{1}{m} \cdot \sum_{i=1}^m \text{Encode}(\mathbf{x}^{(i)})(k). \quad (2.100)$$

Also, let $\mathbf{q}$ be the expected code, where $q_k = \tau$ for any k . If each entry of the average code is viewed as a Bernoulli random variable with mean $\bar{h}_k$, and each entry of the expected code is viewed as another Bernoulli random variable with mean τ , then the regularization term can be defined as the sum of the KL divergence between $\bar{h}_k$ and q_k over all entries:

$$\begin{aligned} R &= \sum_k D_{\text{KL}}(q_k \| \bar{h}_k) \\ &= \sum_k \tau \cdot \log \frac{\tau}{\bar{h}_k} + (1 - \tau) \cdot \log \frac{1 - \tau}{1 - \bar{h}_k}. \end{aligned} \quad (2.101)$$

In this form, R penalizes the model when $\bar{\mathbf{h}}$ deviates from $\mathbf{q}$.

As another auto-encoder variant, the **contractive auto-encoder (CAE)** tries to improve the robustness of representation by introducing a new regularization term into training [Rifai et al., 2011]:

$$\begin{aligned} R &= \sum_{i=1}^m \left\| \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{x}^{(i)}} \right\|_F^2 \\ &= \sum_{i=1}^m \left\| \frac{\partial \text{Encode}(\mathbf{x}^{(i)})}{\partial \mathbf{x}^{(i)}} \right\|_F^2, \end{aligned} \quad (2.102)$$

where $\frac{\partial \text{Encode}(\mathbf{x}^{(i)})}{\partial \mathbf{x}^{(i)}}$ (or $\frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{x}^{(i)}}$) is the **Jacobian matrix** of the representation¹³, and $\|\cdot\|_F$ is the **Frobenius norm** of a matrix¹⁴. The contractive penalty helps resist the influence of

¹³Suppose that the encoder is a function $\text{Encode}(\cdot): \mathbf{x} \in \mathbb{R}^{d_x} \rightarrow \mathbf{h} \in \mathbb{R}^{d_h}$. The Jacobian matrix of $\mathbf{h} = \text{Encode}(\mathbf{x})$ is a $d_h \times d_x$ matrix:

$$\begin{aligned} \text{Jacobian} &= \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \\ &= \begin{bmatrix} \frac{\partial \mathbf{h}}{\partial x_1} & \cdots & \frac{\partial \mathbf{h}}{\partial x_{d_x}} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_{d_x}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{d_h}}{\partial x_1} & \cdots & \frac{\partial h_{d_h}}{\partial x_{d_x}} \end{bmatrix}. \end{aligned} \quad (2.103)$$

¹⁴For a matrix $\mathbf{A} \in \mathbb{R}^{d_x \times d_h}$, the Frobenius norm is essentially the l_2 norm of its flattened vector representation, given by the equation:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}. \quad (2.104)$$

Note that in strict linear algebra, the l_2 norm of a matrix refers to its maximum singular value (the spectral norm), which is why the term Frobenius norm is explicitly used here to describe the element-wise magnitude.

small perturbations to the input. In the geometric sense, it encourages that the neighborhood relationship holds for output data points if the input data points are close neighbors, in other words, it forces $\text{Encode}(\cdot)$ to behave more like a **contraction mapping**¹⁵, hence the name of contractive auto-encoder.

2.6.2 Denoising Auto-encoders

Another source of inspiration for improving the robustness of a model arises from the **denoising** idea: we add noise to the input and then remove it to recover the clean data. **Denoising auto-encoders (DAEs)** are a class of neural networks that combine the principles of auto-encoding and denoising. First, noise is added to the input vector in a stochastic manner. This can be described as a process of generating a noisy input $\mathbf{x}_{\text{noise}} \in \mathbb{R}^{d_x}$ given the original input $\mathbf{x} \in \mathbb{R}^{d_x}$:

$$\mathbf{x}_{\text{noise}} \sim \text{Pr}_{\text{noise}}(\mathbf{x}_{\text{noise}}|\mathbf{x}), \quad (2.106)$$

where $\text{Pr}_{\text{noise}}(\cdot|\mathbf{x})$ is a conditional distribution for sampling $\mathbf{x}_{\text{noise}}$. For example, we can follow the method presented in Section 2.5.5 and draw the noisy input from a multivariate Gaussian distribution:

$$\mathbf{x}_{\text{noise}} \sim \mathcal{N}(\mathbf{x}, \sigma^2 \mathbf{I}), \quad (2.107)$$

where $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ denotes a Gaussian distribution with mean vector $\boldsymbol{\mu} \in \mathbb{R}^{d_x}$ and covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{d_x \times d_x}$. Equivalently, this additive noise can be explicitly formulated by isolating the zero-mean noise term:

$$\mathbf{x}_{\text{noise}} = \mathbf{x} + \mathbf{g}, \quad (2.108)$$

$$\mathbf{g} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \quad (2.109)$$

where $\sigma^2 \mathbf{I}$ is a diagonal covariance matrix, indicating that independent Gaussian noise with variance σ^2 is applied to each dimension of the input, and $\mathbf{I}$ is the $d_x \times d_x$ identity matrix.

Sometimes, this process is called the **corruption** of the input, and $\mathbf{x}_{\text{noise}}$ is called the **corrupted input**. Aside from additive Gaussian noise, there are a few different ways to corrupt the input [Vincent et al., 2010]. One of the popular methods is to zero some of the entries of $\mathbf{x}$. For example, we can set each entry to 0 with a pre-defined probability. This is also called **masking noise**. Another method is to use **salt-and-pepper noise** or **impulse noise** for corruption. It randomly chooses some of the entries, and sets each of them to a minimum or maximum value with a pre-defined probability. Different types of noise are applied to different applications of auto-encoders. For example, the masking noise is popular in training language

¹⁵Let X be a metric space with a metric d . Given a function $f(\cdot)$ from X to X , $f(\cdot)$ is a *contraction mapping* if there exists a constant $\epsilon \in [0, 1)$ such that for any $x_1, x_2 \in X$:

$$d(f(x_1), f(x_2)) \leq \epsilon \cdot d(x_1, x_2). \quad (2.105)$$

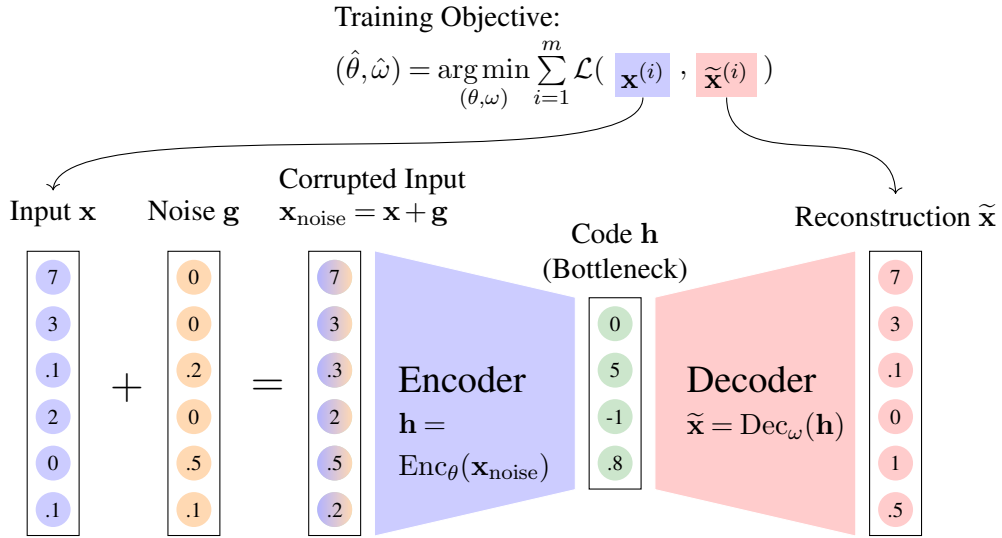


Figure 2.17: Illustration of a denoising auto-encoder. An input $\mathbf{x}$ is first corrupted into a noisy or corrupted input $\mathbf{x}_{\text{noise}}$. Then, it is passed through an encoder to form a code $\mathbf{h}$. Then, the code is passed through a decoder to form a reconstructed input $\tilde{\mathbf{x}}$. The training is performed by minimizing the loss between $\mathbf{x}$ and $\tilde{\mathbf{x}}$. This process is termed “denoising” because it tries to remove the noise from $\mathbf{x}_{\text{noise}}$ and recover the original input $\mathbf{x}$.

models, and the salt-and-pepper noise is more commonly used in image processing.

Then, the corrupted input $\mathbf{x}_{\text{noise}}$ is fed into an encoder-decoder network, and the network produces a reconstructed input $\tilde{\mathbf{x}} = \text{Decode}(\text{Encode}(\mathbf{x}_{\text{noise}}))$. The training process is regular. We reuse Eq. (2.95) to minimize the loss of replacing $\mathbf{x}$ with $\tilde{\mathbf{x}}$. Thus, we can rewrite Eq. (2.95) to adapt the objective to the denoising case:

$$(\hat{\theta}, \hat{\omega}) = \arg \min_{(\theta, \omega)} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, \text{Decode}_{\omega}(\text{Encode}_{\theta}(\mathbf{x}_{\text{noise}}^{(i)}))). \quad (2.110)$$

Eq. (2.110) differs from Eq. (2.95) only in that the input of the auto-encoder is $\mathbf{x}_{\text{noise}}$ instead of $\mathbf{x}$. In other words, we denoise the corrupted input to recover the original input. See Figure 2.17 for the structure of denoising auto-encoders.

Note that both contractive/sparse auto-encoders and denoising auto-encoders can be thought of as ways to improve the robustness of auto-encoders. Their difference lies in that they regularize the training at different points of the model. Contractive auto-encoders aim at improving the robustness of encoding, that is, the representation is trained to be not so sensitive to small perturbations to the input. Denoising auto-encoders, on the other hand, aim at improving the robustness of reconstruction. They affect both encoders and decoders simultaneously. In some sense, denoising auto-encoders are direct applications of noisy training to auto-encoders (see Section 2.5.5). It is of course difficult to say which models are better. For example, contractive auto-encoders have more direct guidance on learning the representation,

which is often our primary objective in feature learning. The training of denoising auto-encoders, although it has an indirect effect on encoding, receives additional denoising signals from the decoder. This offers a new view of robust training: a robust representation can be learned at both its point of generation (the denoising encoder) and its point of application (the denoising decoder).

2.6.3 Variational Auto-encoders

Despite the name, **variational auto-encoders (VAEs)** were not originally proposed to solve standard representation encoding problems. They are fundamentally designed to generate new data similar to observed data, hence possessing very different mathematical formulations from the classical auto-encoders discussed above. In statistics and machine learning, variational auto-encoders are more often viewed as instances of variational Bayesian methods and used to perform efficient statistical inference over latent variables when the posterior probabilities of these variables are intractable [Kingma and Welling, 2014; 2019]. On the other hand, variational auto-encoders, implicitly or explicitly, deal with what we do in inducing the underlying representation of an observed object. We therefore include them in this section for a relatively complete discussion.

We begin with a generative story describing how each data point is generated. Suppose that, for an observed sample $\mathbf{x}$ in our dataset, there is an unobserved latent variable $\mathbf{h}$ that describes $\mathbf{x}$. Now we intend to develop a probabilistic model to describe the generation process of $\mathbf{x}$, specifically, estimating the probability $\Pr(\mathbf{x})$. This can be computed as the marginal distribution:

$$\Pr(\mathbf{x}) = \int \Pr(\mathbf{x}, \mathbf{h}) d\mathbf{h}, \quad (2.111)$$

where we explicitly introduce the latent variable $\mathbf{h}$ into the inference of $\mathbf{x}$. To solve Eq. (2.111), we use a model $p_\omega(\mathbf{x}, \mathbf{h})$ to approximate $\Pr(\mathbf{x}, \mathbf{h})$ (i.e., $p_\omega(\mathbf{x}, \mathbf{h}) \approx \Pr(\mathbf{x}, \mathbf{h})$), and we have

$$p_\omega(\mathbf{x}) = \int p_\omega(\mathbf{x}, \mathbf{h}) d\mathbf{h}, \quad (2.112)$$

where $p_\omega(\mathbf{x}, \mathbf{h})$ is a probability density function parameterized by ω . We replace the left-hand side of Eq. (2.112) with $p_\omega(\mathbf{x})$ to emphasize that the probability is determined by the model $p_\omega(\cdot)$. There are generally many ways to define $p_\omega(\mathbf{x}, \mathbf{h})$. Here we can simply think of it as a neural network.

Then, we can rewrite Eq. (2.112) by using the chain rule:

$$p_\omega(\mathbf{x}) = \int p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h}) d\mathbf{h}, \quad (2.113)$$

where $p_\omega(\mathbf{h})$ is the prior over $\mathbf{h}$, e.g., a Gaussian prior. The conditional probability $p_\omega(\mathbf{x}|\mathbf{h})$ describes how likely $\mathbf{x}$ is observed given the latent variable $\mathbf{h}$. To model this generation process, $p_\omega(\mathbf{x}|\mathbf{h})$ is often assumed to be a Gaussian distribution that is parameterized with its

mean $\boldsymbol{\mu}_p$ and variance σ_p :

$$p_\omega(\mathbf{x}|\mathbf{h}) = \mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2 \mathbf{I}), \quad (2.114)$$

where $\boldsymbol{\mu}_p \in \mathbb{R}^{d_x}$ and $\sigma_p \in \mathbb{R}$ are determined by a decoding network $\text{Decode}_\omega(\cdot)$ (we will explain later on why it is called “decoding”):

$$(\boldsymbol{\mu}_p, \sigma_p) = \text{Decode}_\omega(\mathbf{h}). \quad (2.115)$$

However, the marginal likelihood in Eq. (2.112) remains intractable. Although we can easily evaluate the prior $p_\omega(\mathbf{h})$ and the conditional $p_\omega(\mathbf{x}|\mathbf{h})$, performing the exact integration over the entire continuous latent space of $\mathbf{h}$ is computationally prohibitive. This also leads to an intractable posterior:

$$p_\omega(\mathbf{h}|\mathbf{x}) = \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{x})}. \quad (2.116)$$

It looks like we are stuck with $p_\omega(\mathbf{x})$ and $p_\omega(\mathbf{h}|\mathbf{x})$! Variational auto-encoders address this issue by approximating $p_\omega(\mathbf{h}|\mathbf{x})$ with a tractable posterior $q_\theta(\mathbf{h}|\mathbf{x})$:

$$q_\theta(\mathbf{h}|\mathbf{x}) \approx p_\omega(\mathbf{h}|\mathbf{x}), \quad (2.117)$$

where θ is the parameter of the new model. Like Eqs. (2.114-2.115), $q_\theta(\mathbf{h}|\mathbf{x})$ is defined as another Gaussian distribution:

$$q_\theta(\mathbf{h}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2 \mathbf{I}), \quad (2.118)$$

$$(\boldsymbol{\mu}_q, \sigma_q) = \text{Encode}_\theta(\mathbf{x}), \quad (2.119)$$

where $\text{Encode}_\theta(\cdot)$ is the encoding network that reads $\mathbf{x}$ and generates the mean and variance of the distribution $q_\theta(\mathbf{h}|\mathbf{x})$. This is interesting! We now have a feasible path to compute $\Pr(\mathbf{x})$: we can sample a latent variable $\mathbf{h}$ from the variational posterior $q_\theta(\mathbf{h}|\mathbf{x})$, and then evaluate the joint probability $p_\omega(\mathbf{x}, \mathbf{h})$. Mathematically, we can reformulate the log-likelihood of the observation as an expectation over the variational posterior:

$$\begin{aligned} & \log p_\omega(\mathbf{x}) \\ &= \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} [\log p_\omega(\mathbf{x})] \\ &= \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})} \right] \\ &= \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{p_\omega(\mathbf{h}|\mathbf{x})} \cdot \frac{q_\theta(\mathbf{h}|\mathbf{x})}{q_\theta(\mathbf{h}|\mathbf{x})} \right] \\ &= \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log \frac{q_\theta(\mathbf{h}|\mathbf{x})}{p_\omega(\mathbf{h}|\mathbf{x})} \right] + \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})} \right]. \end{aligned} \quad (2.120)$$

The first term on the right-hand side of Eq. (2.120) is precisely the KL divergence (or

relative entropy) between the approximate posterior $q_\theta(\mathbf{h}|\mathbf{x})$ and the true posterior $p_\omega(\mathbf{h}|\mathbf{x})$:

$$D_{\text{KL}}(q_\theta(\mathbf{h}|\mathbf{x}) \| p_\omega(\mathbf{h}|\mathbf{x})) = \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log \frac{q_\theta(\mathbf{h}|\mathbf{x})}{p_\omega(\mathbf{h}|\mathbf{x})} \right]. \quad (2.121)$$

Since the KL divergence is strictly non-negative ($D_{\text{KL}}(q \| p) \geq 0$, and equals zero if and only if $q = p$), we can establish a strict lower bound on the log-likelihood:

$$\begin{aligned} \log p_\omega(\mathbf{x}) &\geq \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log \frac{p_\omega(\mathbf{h}) \cdot p_\omega(\mathbf{x}|\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})} \right] \\ &= \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} \left[\log p_\omega(\mathbf{x}|\mathbf{h}) + \log \frac{p_\omega(\mathbf{h})}{q_\theta(\mathbf{h}|\mathbf{x})} \right] \\ &= \mathbb{E}_{\mathbf{h} \sim q_\theta(\mathbf{h}|\mathbf{x})} [\log p_\omega(\mathbf{x}|\mathbf{h})] - D_{\text{KL}}(q_\theta(\mathbf{h}|\mathbf{x}) \| p_\omega(\mathbf{h})). \end{aligned} \quad (2.122)$$

The right-hand side of Eq. (2.122) constitutes the foundational **evidence lower bound (ELBO)** of the data likelihood. The first term of the ELBO (the expected reconstruction log-likelihood) can be approximated using Monte Carlo sampling of $\mathbf{h}$. The second term is computationally straightforward because an analytical solution exists for the KL divergence between two Gaussian distributions. Let $\mathcal{L}(\mathbf{x}, \theta, \omega)$ denote the negative ELBO. The training process of a variational auto-encoder is then framed as minimizing $\mathcal{L}(\cdot)$ over a dataset of observed samples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$:

$$(\hat{\theta}, \hat{\omega}) = \arg \min_{(\theta, \omega)} \sum_{i=1}^m \mathcal{L}(\mathbf{x}^{(i)}, \theta, \omega). \quad (2.123)$$

Note that because sampling $\mathbf{h}$ from $q_\theta(\mathbf{h} | \mathbf{x})$ is a stochastic, non-differentiable operation, its gradient cannot be computed directly. To fit the training of auto-encoders in standard back-propagation, a common way is to use the so-called **reparameterization trick**. Here we skip the details and refer the reader to a few papers for more information [Kingma and Welling, 2014; Doersch, 2016].

Figure 2.18 illustrates how a variational auto-encoder works. It presents a two-step generation process:

- **Encoding.** For an input sample $\mathbf{x}$, we sample a latent variable $\mathbf{h}$ from $q_\theta(\mathbf{h}|\mathbf{x})$. This involves an encoding network $\text{Encode}_\theta(\mathbf{x})$ that generates the mean $\boldsymbol{\mu}_q$ and variance σ_q^2 of the Gaussian distribution $q_\theta(\mathbf{h}|\mathbf{x})$. The latent variable is then generated by sampling from $\mathcal{N}(\boldsymbol{\mu}_q, \sigma_q^2 \mathbf{I})$.
- **Decoding.** For the latent variable $\mathbf{h}$, we sample the original input $\mathbf{x}$ from $p_\omega(\mathbf{x}|\mathbf{h})$. It follows again a Gaussian sampling process: a decoding network $\text{Decode}_\omega(\mathbf{h})$ is used to determine the mean $\boldsymbol{\mu}_p$ and variance σ_p^2 of the distribution. $\mathbf{x}$ is generated by following $\mathcal{N}(\boldsymbol{\mu}_p, \sigma_p^2 \mathbf{I})$.

Sometimes, $q_\theta(\mathbf{h}|\mathbf{x})$ and $p_\omega(\mathbf{x}|\mathbf{h})$ are called an “encoder” and a “decoder”, as they try to “map” an input to a representation and then “map” it back to the input. However, $q_\theta(\mathbf{h}|\mathbf{x})$ and $p_\omega(\mathbf{x}|\mathbf{h})$ themselves imply some non-deterministic models, that is, they output the probability

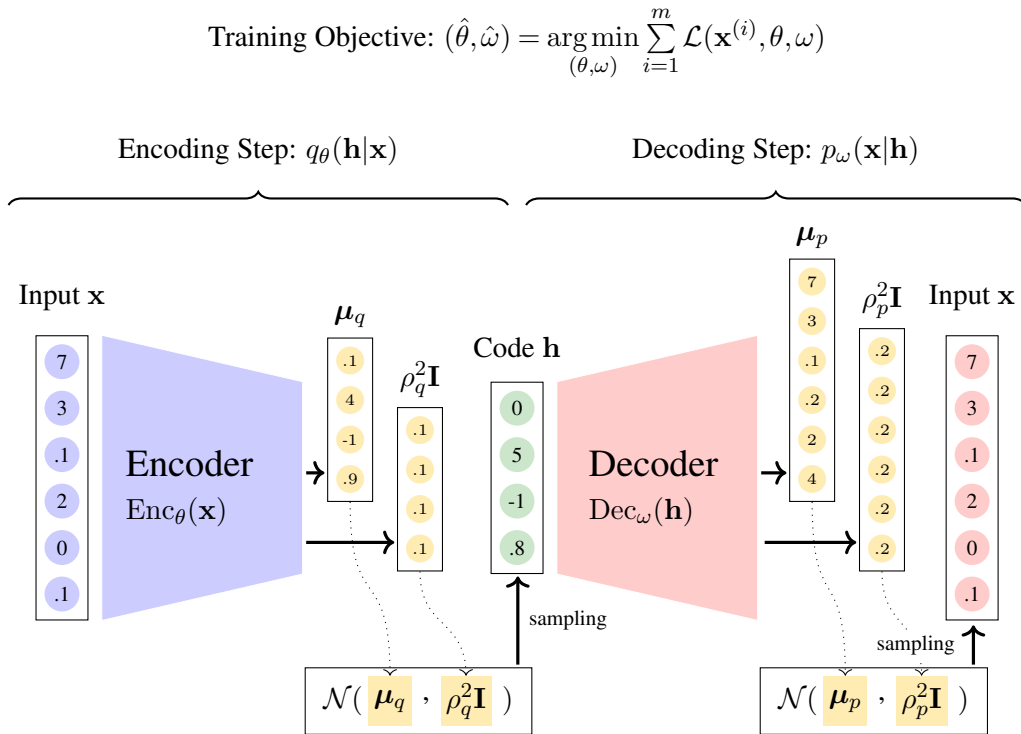


Figure 2.18: Illustration of a variational auto-encoder. For an input sample $\mathbf{x}$, we generate a latent variable $\mathbf{h}$ by using an encoder $q_{\theta}(\mathbf{h}|\mathbf{x})$. In the encoding step, a neural network $\text{Enc}_{\theta}(\cdot)$ is first used to produce the mean and variance of a Gaussian distribution, i.e., μ_q and σ_q^2 . The latent variable $\mathbf{h}$ is then drawn according to $\mathcal{N}(\mu_q, \sigma_q^2 \mathbf{I})$. After that, we regenerate the original sample $\mathbf{x}$ by using a decoder $p_{\omega}(\mathbf{x}|\mathbf{h})$. In the decoding step, like the generation process in the encoder, a neural network $\text{Dec}_{\omega}(\cdot)$ is used to generate the mean μ_p and variance σ_p^2 of $\mathcal{N}(\mu_p, \sigma_p^2 \mathbf{I})$. The same input $\mathbf{x}$ is generated by sampling from $\mathcal{N}(\mu_p, \sigma_p^2 \mathbf{I})$.

density functions of the variables rather than point estimates. An important consequence of this result is that variational auto-encoders do not tend to find the “best” representation for the input. At first glance, this might appear counterintuitive, as conventional models typically yield fixed-value representations. This, however, is the case of the Bayesian inference — we only learn a distribution over possible values of a latent variable. From a practical perspective, if a deterministic representation is required for downstream tasks, one can simply use the expected value (i.e., the mean μ_q) of the inferred distribution, which reliably captures the essential features of the input [Knight, 2009].

In practice, the main use of variational auto-encoders is in generation but not representation. At test time, provided the optimized parameters $\hat{\theta}$ and $\hat{\omega}$, the encoder (i.e., $q_{\hat{\theta}}(\mathbf{h}|\mathbf{x})$) is removed, and the decoder (i.e., $p_{\hat{\omega}}(\mathbf{x}|\mathbf{h})$) works with randomly generated $\mathbf{h}$ ’s. More precisely, we sample a latent variable $\mathbf{h}_{\text{new}}$ from a Gaussian distribution, and infer a sample $\mathbf{x}_{\text{new}}$ by $p_{\hat{\omega}}(\mathbf{x}|\mathbf{h}_{\text{new}})$ as usual. We will see in the subsequent chapters that many NLP problems can be categorized as generation problems where sequential or hierarchical data objects are generated on the

condition of some given data objects or latent variables.

2.7 Summary

In this chapter, we have discussed what a neural network is, as well as a few basic architectures that are commonly used as building blocks for developing powerful deep learning systems. Furthermore, we have examined how to train neural networks, regularize the training process, and apply neural networks to feature learning in an unsupervised manner.

However, neural networks and deep learning are wide-ranging topics, and our discussions merely scratch the surface. For a more comprehensive introduction, [Goodfellow et al. \[2016\]](#)'s book is an excellent resource. It also covers several advanced techniques, such as deep structured models and randomized methods, for developing state-of-the-art systems. As always, there is a significant difference between understanding a technique theoretically and being proficient in applying it to solve real-world problems. Therefore, for practitioners who wish to apply neural networks and deep learning in various scenarios, there are many books detailing the implementation of deep learning systems [[Géron, 2019](#); [Zhang et al., 2021](#); [Chollet, 2021](#)], along with open-source projects that provide reference codebases¹⁶.

In the following chapters, we will focus on how to use neural models to address NLP problems. Along the way, we will explore how to learn the representations of words and sentences using the methods discussed so far (Chapters 3-4), and how to model various NLP problems using several advanced neural network-based methods. These include the attention mechanism and Transformers (Chapters 5-6), pre-training (Chapter 7), and large language models (Chapters 8-11).

¹⁶URLs to a few popular online tutorials: <https://pytorch.org/tutorials>, <https://keras.io/examples/nlp>, and <https://www.tensorflow.org/tutorials>

Bibliography

- [Ackley et al., 1985] David H Ackley, Geoffrey E Hinton, and Terrence J Sejnowski. A learning algorithm for boltzmann machines. *Cognitive science*, 9(1):147–169, 1985.
- [Alzantot et al., 2018] Moustafa Alzantot, Yash Sharma, Ahmed Elgohary, Bo-Jhang Ho, Mani Srivastava, and Kai-Wei Chang. Generating natural language adversarial examples. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2890–2896, 2018.
- [Ba et al., 2016] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [Belinkov and Bisk, 2018] Yonatan Belinkov and Yonatan Bisk. Synthetic and natural noise both break neural machine translation. In *International Conference on Learning Representations*, 2018.
- [Bengio et al., 2000] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. A neural probabilistic language model. *Advances in Neural Information Processing Systems*, 13, 2000.
- [Bengio et al., 2003] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3(Feb):1137–1155, 2003.
- [Bickel and Doksum, 2015] Peter J Bickel and Kjell A Doksum. *Mathematical statistics: basic ideas and selected topics, volumes I-II package*. Chapman and Hall/CRC, 2015.
- [Bishop, 1995] Christopher Bishop. Regularization and complexity control in feed-forward networks. In *Proceedings International Conference on Artificial Neural Networks ICANN'95*, pages 141–148, 1995a.
- [Bishop, 1995] Christopher M. Bishop. Training with noise is equivalent to tikhonov regularization. *Neural computation*, 7(1):108–116, 1995b.
- [Chiang and Cholak, 2022] David Chiang and Peter Cholak. Overcoming a theoretical limitation of self-attention. *arXiv preprint arXiv:2202.12172*, 2022.
- [Chollet, 2021] François Chollet. *Deep Learning with Python (2nd ed.)*. Manning Publications, 2021.
- [Cybenko, 1989] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.
- [Dayan et al., 1995] Peter Dayan, Geoffrey E Hinton, Radford M Neal, and Richard S Zemel. The helmholtz machine. *Neural computation*, 7(5):889–904, 1995.
- [Doersch, 2016] Carl Doersch. Tutorial on variational autoencoders. *stat*, 1050:13, 2016.
- [Duchi et al., 2011] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- [Ebrahimi et al., 2018] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. HotFlip: White-box

- adversarial examples for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 31–36, Melbourne, Australia, 2018.
- [Elman, 1990] Jeffrey L Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.
- [Feng et al., 2021] Steven Y. Feng, Varun Gangal, Jason Wei, Sarath Chandar, Soroush Vosoughi, Teruko Mitamura, and Eduard Hovy. A survey of data augmentation approaches for NLP. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 968–988, 2021.
- [Géron, 2019] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems (2nd ed.)*. O’Reilly Media, 2019.
- [Glorot and Bengio, 2010] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [Goodfellow et al., 2015] Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.
- [Goodfellow et al., 2016] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [Goodman, 1996] Joshua Goodman. Parsing algorithms and metrics. In *34th Annual Meeting of the Association for Computational Linguistics*, pages 177–183, 1996.
- [Graves et al., 2013] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. Speech recognition with deep recurrent neural networks. In *2013 IEEE international conference on acoustics, speech and signal processing*, pages 6645–6649. IEEE, 2013.
- [Gulcehre et al., 2016] Caglar Gulcehre, Marcin Moczulski, Misha Denil, and Yoshua Bengio. Noisy activation functions. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 3059–3068. PMLR, 2016.
- [He et al., 2015] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.
- [He et al., 2016] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [Hinton, 2018] Geoff Hinton. Coursera neural networks for machine learning lecture 6, 2018. URL http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
- [Hinton, 2007] Geoffrey E Hinton. Learning multiple layers of representation. *Trends in cognitive sciences*, 11(10):428–434, 2007.
- [Hinton and Salakhutdinov, 2006] Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.
- [Hinton et al., 2006] Geoffrey E Hinton, Simon Osindero, and Yee-Whye Teh. A fast learning algorithm for deep belief nets. *Neural computation*, 18(7):1527–1554, 2006.

- [Hinton et al., 2012] Geoffrey E Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.
- [Hochreiter and Schmidhuber, 1997] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [Holmström and Koistinen, 1992] Lasse Holmström and Petri Koistinen. Using additive noise in back-propagation training. *IEEE Transactions on Neural Networks*, 3(1):24–38, 1992.
- [Hopfield, 1982] John J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the national academy of sciences*, 79(8):2554–2558, 1982.
- [Hopfield, 1984] John J Hopfield. Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the national academy of sciences*, 81(10):3088–3092, 1984.
- [Huang et al., 2020] Xiao Shi Huang, Felipe Perez, Jimmy Ba, and Maksims Volkovs. Improving transformer optimization through better initialization. In *Proceedings of International Conference on Machine Learning*, pages 4475–4483. PMLR, 2020.
- [Hubel and Wiesel, 1959] David H Hubel and Torsten N Wiesel. Receptive fields of single neurones in the cat’s striate cortex. *The Journal of physiology*, 148(3):574, 1959.
- [Ioffe and Szegedy, 2015] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.
- [Jia and Liang, 2017] Robin Jia and Percy Liang. Adversarial examples for evaluating reading comprehension systems. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2021–2031, 2017.
- [Kingma and Ba, 2014] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [Kingma and Welling, 2014] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *Proceedings of 2nd International Conference on Learning Representations, ICLR 2014*, 2014.
- [Kingma and Welling, 2019] Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 2019.
- [Knight, 2009] Kevin Knight. Bayesian inference with tears, 2009.
- [Kochenderfer and Wheeler, 2019] Mykel J. Kochenderfer and Tim A. Wheeler. *Algorithms for Optimization*. The MIT Press, 2019.
- [Kumar and Byrne, 2004] Shankar Kumar and William Byrne. Minimum Bayes-risk decoding for statistical machine translation. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics: HLT-NAACL 2004*, pages 169–176, 2004.
- [LeCun et al., 1989] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.
- [LeCun et al., 2012] Yann A LeCun, Léon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural networks: Tricks of the trade*, pages 9–48. Springer, 2012.

- [Liu et al., 2020] Liyuan Liu, Xiaodong Liu, Jianfeng Gao, Weizhu Chen, and Jiawei Han. Understanding the difficulty of training transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5747–5763, November 2020.
- [McCulloch and Pitts, 1943] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, 1943.
- [Minsky and Papert, 1969] Marvin Minsky and Seymour Papert. *Perceptrons*. MIT press, 1969.
- [Mohri et al., 2018] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning (2nd ed.)*. MIT Press, 2018.
- [Nair and Hinton, 2009] Vinod Nair and Geoffrey E Hinton. 3d object recognition with deep belief nets. *Advances in neural information processing systems*, 22, 2009.
- [Neelakantan et al., 2015] Arvind Neelakantan, Luke Vilnis, Quoc V Le, Ilya Sutskever, Lukasz Kaiser, Karol Kurach, and James Martens. Adding gradient noise improves learning for very deep networks. *arXiv preprint arXiv:1511.06807*, 2015.
- [Nesterov, 1983] Yurii E Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. In *Dokl. akad. nauk Sssr*, volume 269, pages 543–547, 1983.
- [Olive, 2022] David Olive. Robust statistics, 2022. URL <http://parker.ad.siu.edu/Olive/ol-bookp.htm>.
- [Olshausen and Field, 1997] Bruno A Olshausen and David J Field. Sparse coding with an overcomplete basis set: A strategy employed by v1? *Vision research*, 37(23):3311–3325, 1997.
- [Ott et al., 2018] Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. Scaling neural machine translation. In *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 1–9, October 2018.
- [Pearson, 1901] Karl Pearson. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin philosophical magazine and journal of science*, 2(11):559–572, 1901.
- [Plaut et al., 1986] David C Plaut, Steven J Nowlan, and Geoffrey E Hinton. Experiments on learning by back propagation. Technical report, Carnegie-Mellon University, 1986.
- [Polyak, 1964] Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *Ussr computational mathematics and mathematical physics*, 4(5):1–17, 1964.
- [Prechelt, 1998] Lutz Prechelt. Early stopping-but when? In *Neural Networks: Tricks of the trade*, pages 55–69. Springer, 1998.
- [Rifai et al., 2011] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In *Proceedings of International Conference on Machine Learning*, 2011.
- [Rosenblatt, 1957] Frank Rosenblatt. *The perceptron, a perceiving and recognizing automaton Project Para*. Cornell Aeronautical Laboratory, 1957.
- [Rumelhart et al., 1986] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- [Sennrich et al., 2016] Rico Sennrich, Barry Haddow, and Alexandra Birch. Improving neural machine translation models with monolingual data. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 86–96, 2016.

- [Shorten and Khoshgoftaar, 2019] Connor Shorten and Taghi M Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of big data*, 6(1):1–48, 2019.
- [Smith et al., 2018] Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. Don’t decay the learning rate, increase the batch size. In *Proceedings of the 6th International Conference on Learning Representations ICLR*, 2018.
- [Srivastava et al., 2014] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.
- [Sutskever et al., 2013] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *International conference on machine learning*, pages 1139–1147. PMLR, 2013.
- [Szegedy et al., 2014] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *Proceedings of the 2nd International Conference on Learning Representations*, 2014.
- [Szegedy et al., 2016] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.
- [Uffink, 2017] Jos Uffink. Boltzmann’s Work in Statistical Physics. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Spring 2017 edition, 2017.
- [Ulyanov et al., 2016] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.
- [Vaswani et al., 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of Advances in Neural Information Processing Systems*, volume 30, 2017.
- [Vincent et al., 2010] Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of machine learning research*, 11(12), 2010.
- [Waibel et al., 1989] Alex Waibel, Toshiyuki Hanazawa, Geoffrey Hinton, Kiyohiro Shikano, and Kevin J Lang. Phoneme recognition using time-delay neural networks. *IEEE transactions on acoustics, speech, and signal processing*, 37(3):328–339, 1989.
- [Williams and Zipser, 1989] Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.
- [Wold et al., 1987] Svante Wold, Kim Esbensen, and Paul Geladi. Principal component analysis. *Chemometrics and intelligent laboratory systems*, 2(1-3):37–52, 1987.
- [Wu and He, 2018] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*, pages 3–19, 2018.
- [Xu et al., 2020] Hongfei Xu, Qiuhui Liu, Josef van Genabith, Deyi Xiong, and Jingyi Zhang. Lipschitz constrained parameter initialization for deep transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 397–402, July 2020.
- [Yao et al., 2007] Yuan Yao, Lorenzo Rosasco, and Andrea Caponnetto. On early stopping in gradient descent learning. *Constructive Approximation*, 26:289–315, 2007.

-
- [Zeiler, 2012] Matthew D Zeiler. Adadelata: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*, 2012.
- [Zhang et al., 2021] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. Dive into deep learning. *arXiv preprint arXiv:2106.11342*, 2021.
- [Zhang et al., 2020] Wei Emma Zhang, Quan Z Sheng, Ahoud Alhazmi, and Chenliang Li. Adversarial attacks on deep-learning models in natural language processing: A survey. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 11(3):1–41, 2020.